HW04

ENAE 601: Astrodynamics

October 21, 2026 at 23:59:00

Dr. Healy

Vai Srivastava

Problem 1

Curtis 2.45

Relative to an earth-centered, nonrotating frame, the position and velocity vectors of a spacecraft are $\mathbf{r}_0 = 6320\hat{\mathbf{i}} + 7750\hat{\mathbf{k}}$ (km) and $\mathbf{v}_0 = 11\hat{\mathbf{j}}$ (km s^{-1}). Use the $\Delta\theta$ Lagrange coefficient method.

a) Find the position vector 10 min later.

b) Calculate the change in true anomaly over the 10 min time span.

a) With $\mu = 398\,600\,\text{km}^3\,\text{s}^{-2}$, the initial radius and radial velocity are:

$$r_0 = \sqrt{6320^2 + 7750^2}\ \text{km} = 10000.244\,997\,\text{km},$$
$$v_{r_0} = \frac{\mathbf{r}_0 \cdot \mathbf{v}_0}{r_0} = 0.$$

The angular momentum, eccentricity vector, and specific energy are:

$$\mathbf{h} = \mathbf{r}_0 \times \mathbf{v}_0 = [-85250, 0, 69520]\ (\text{km}^2\,\text{s}^{-1}), \quad h = 110\,002.694\,967\,\text{km}^2\,\text{s}^{-1},$$
$$\mathbf{e} = \frac{\mathbf{v}_0 \times \mathbf{h}}{\mu} - \frac{\mathbf{r}_0}{r_0} = [1.2865302852, 0, 1.5776281188], \quad e = 2.0356990583,$$
$$\mathcal{E} = \frac{v_0^2}{2} - \frac{\mu}{r_0} = 20.640\,977\,\text{km}^2\,\text{s}^{-2}.$$

Since $e > 1$ and $\mathcal{E} > 0$, the trajectory is a hyperbola. Furthermore, $v_{r_0} = 0$ and $\mathbf{e}$ is parallel to $\mathbf{r}_0$, so the spacecraft is at perigee at $t = 0$, meaning $\theta_0 = 0$.

The $\Delta\theta$ Lagrange coefficients require the true anomaly swept in the 10 min coast, which we obtain from Kepler's equation for the hyperbola. With $t = 600\,\text{s}$ measured from perigee, the hyperbolic mean anomaly is:

$$M_h = \frac{\mu^2}{h^3}(e^2 - 1)^{\frac{3}{2}} t = 0.3992586277.$$

Solving $e\sinh(F) - F = M_h$ for the hyperbolic eccentric anomaly F via Newton's method, starting at $F_0 = M_h$:

$$F_{k+1} = F_k - \frac{e\sinh(F_k) - F_k - M_h}{e\cosh(F_k) - 1},$$

converges to $F = 0.3689340897$. Converting to true anomaly,

$$\tan\left(\frac{\theta}{2}\right) = \sqrt{\frac{e+1}{e-1}}\tanh\left(\frac{F}{2}\right),$$
$$\theta = 34.685\,070°,$$
$$\Delta\theta = \theta - \theta_0 = 34.685\,070°.$$

The orbit equation in terms of $\Delta\theta$ then gives the radius at $t = 10\,\text{min}$:

$$r = \frac{h^2}{\mu}\frac{1}{1 + \left(\frac{h^2}{\mu r_0} - 1\right)\cos(\Delta\theta) - \frac{h v_{r_0}}{\mu}\sin(\Delta\theta)} = 11353.185\,524\,\text{km}.$$

The Lagrange coefficients are:

$$f = 1 - \frac{\mu r}{h^2}(1 - \cos(\Delta\theta)) = 0.9335408947,$$
$$g = \frac{r r_0}{h}\sin(\Delta\theta) = 587.336\,691\,\text{s},$$
$$\dot{f} = \frac{\mu}{h}\frac{1-\cos(\Delta\theta)}{\sin(\Delta\theta)}\left[\frac{\mu}{h^2}(1-\cos(\Delta\theta)) - \frac{1}{r_0} - \frac{1}{r}\right] = -0.000\,206\,198\,5\ \text{s}^{-1},$$
$$\dot{g} = 1 - \frac{\mu r_0}{h^2}(1 - \cos(\Delta\theta)) = 0.9414607174.$$

Thus, the position vector 10 min later is:

$$\begin{aligned}\mathbf{r} &= f\mathbf{r}_0 + g\mathbf{v}_0 \\ &= 5899.978454\hat{\mathbf{i}} + 6460.703599\hat{\mathbf{j}} + 7234.941934\hat{\mathbf{k}}\ \text{(km)}\end{aligned}$$ ■

As a check, $|\mathbf{r}| = 11353.185\,524$ km matches the orbit equation, $f\dot{g} - \dot{f}g = 1.000000000000$, and $\mathbf{v} = \dot{f}\mathbf{r}_0 + \dot{g}\mathbf{v}_0 = [-1.303174, 10.356068, -1.598038]$ (km s^{-1}) conserves both $\mathbf{h}$ and $\mathcal{E}$. A direct RK4 integration of $\ddot{\mathbf{r}} = -\mu\mathbf{r}/r^3$ over the same 600 s also reproduces $\mathbf{r}$ to within 1 mm.

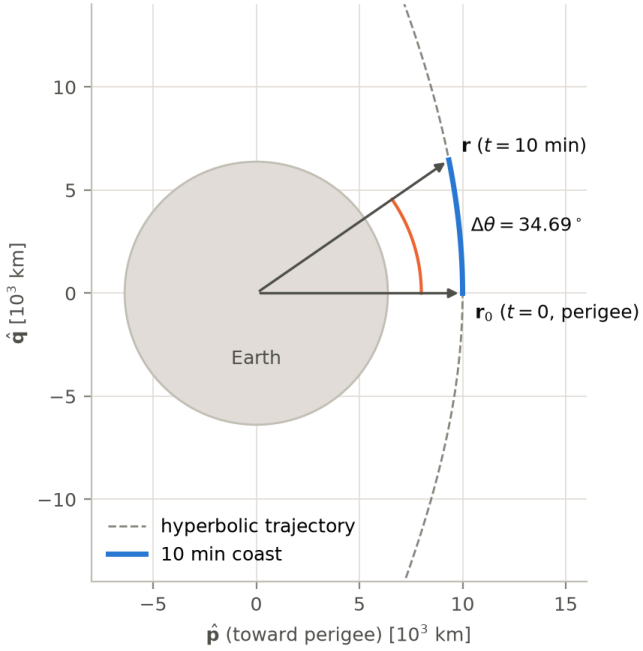


Figure 1: Hyperbolic trajectory in the perifocal frame, with the 10 min coast from perigee highlighted.

b) From part (a), the spacecraft starts at perigee ($\theta_0 = 0$) and reaches $\theta = 34.685\,070°$ after 10 min, so the change in true anomaly is:

$$\Delta\theta = \theta - \theta_0 = 34.685\,070°$$ ■

As an independent check that bypasses Kepler's equation and the Lagrange coefficients entirely, numerically integrating $\ddot{\mathbf{r}} = -\mu\mathbf{r}/r^3$ from $(\mathbf{r}_0, \mathbf{v}_0)$ for 600 s with RK4 gives $\mathbf{r}_{\text{RK4}} = [5899.978454, 6460.703599, 7234.941934]$ (km). The angle between the two position vectors is then:

$$\cos(\Delta\theta) = \frac{\mathbf{r}_0 \cdot \mathbf{r}_{\text{RK4}}}{r_0 |\mathbf{r}_{\text{RK4}}|} = 0.8222923550,$$

and $(\mathbf{r}_0 \times \mathbf{r}_{\text{RK4}}) \cdot \mathbf{h} > 0$, so the spacecraft swept $\Delta\theta = 34.685\,070°$ in the direction of motion, in agreement with the above.

Problem 2

Curtis 7.7

A space station is in an earth orbit with a 90 min period. At $t = 0$, a satellite has the following position and velocity components relative to a CW frame attached to the space station: $\delta\mathbf{r} = 1\hat{\mathbf{i}}$ (km) and $\delta\mathbf{v}_0 = 10\hat{\mathbf{j}}$ ($\mathrm{m\,s^{-1}}$). How far is the satellite from the space station 15 min later?

In the Clohessy–Wiltshire (CW) frame attached to the space station, $\hat{\mathbf{i}}$ is radial, $\hat{\mathbf{j}}$ is along-track, and $\hat{\mathbf{k}}$ is normal to the orbit. The frame rotates at the station's mean motion:

$$n = \frac{2\pi}{T} = \frac{2\pi}{5400\,\mathrm{s}} = 0.001\,163\,552\,8\,\mathrm{rad\,s^{-1}},$$
$$nt = \frac{2\pi}{90\,\mathrm{min}} \cdot 15\,\mathrm{min} = \frac{\pi}{3}.$$

The solution of the CW equations is:

$$\delta\mathbf{r}(t) = \Phi_{rr}(t)\delta\mathbf{r}_0 + \Phi_{rv}(t)\delta\mathbf{v}_0,$$
$$\delta\mathbf{v}(t) = \Phi_{vr}(t)\delta\mathbf{r}_0 + \Phi_{vv}(t)\delta\mathbf{v}_0,$$

with the state transition matrices:

$$\Phi_{rr} = \begin{bmatrix} 4-3\cos(nt) & 0 & 0 \\ 6(\sin(nt)-nt) & 1 & 0 \\ 0 & 0 & \cos(nt) \end{bmatrix}, \qquad \Phi_{rv} = \begin{bmatrix} \frac{\sin(nt)}{n} & \frac{2(1-\cos(nt))}{n} & 0 \\ \frac{2(\cos(nt)-1)}{n} & \frac{4\sin(nt)-3nt}{n} & 0 \\ 0 & 0 & \frac{\sin(nt)}{n} \end{bmatrix},$$

$$\Phi_{vr} = \begin{bmatrix} 3n\sin(nt) & 0 & 0 \\ 6n(\cos(nt)-1) & 0 & 0 \\ 0 & 0 & -n\sin(nt) \end{bmatrix}, \quad \Phi_{vv} = \begin{bmatrix} \cos(nt) & 2\sin(nt) & 0 \\ -2\sin(nt) & 4\cos(nt)-3 & 0 \\ 0 & 0 & \cos(nt) \end{bmatrix}.$$

Here $\delta\mathbf{r}_0 = [1, 0, 0]$ (km) and $\delta\mathbf{v}_0 = [0, 0.010, 0]$ ($\mathrm{km\,s^{-1}}$), so only δx_0 and $\delta\dot{y}_0$ are nonzero. Thus, only the first column of Φ_{rr} and the second column of Φ_{rv} contribute to the position. At $nt = \frac{\pi}{3}$:

$$\begin{aligned} \delta x &= \left(4 - 3\cos\left(\frac{\pi}{3}\right)\right)\delta x_0 + \frac{2\left(1-\cos\left(\frac{\pi}{3}\right)\right)}{n}\delta\dot{y}_0 \\ &= \frac{5}{2}(1\,\mathrm{km}) + (859.436\,693\,\mathrm{s})(0.010\,\mathrm{km\,s^{-1}}) \\ &= 11.094\,367\,\mathrm{km}, \\ \delta y &= 6\left(\sin\left(\frac{\pi}{3}\right) - \frac{\pi}{3}\right)\delta x_0 + \frac{4\sin\left(\frac{\pi}{3}\right) - \pi}{n}\delta\dot{y}_0 \\ &= (-1.087033)(1\,\mathrm{km}) + (277.176\,035\,\mathrm{s})(0.010\,\mathrm{km\,s^{-1}}) \\ &= 1.684\,727\,\mathrm{km}, \\ \delta z &= 0. \end{aligned}$$

Thus, the distance between the satellite and the space station 15 min later is:

$$d = |\delta\mathbf{r}| = \sqrt{\delta x^2 + \delta y^2 + \delta z^2} = 11.221\,554\,\mathrm{km} \quad \blacksquare$$

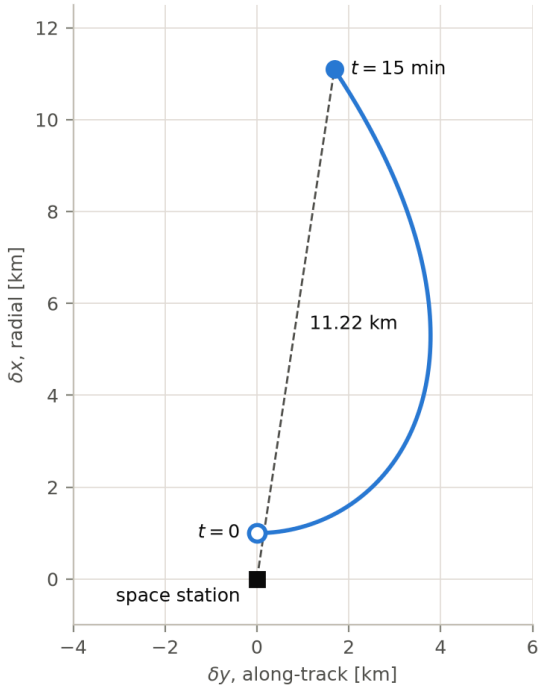


Figure 2: Satellite motion relative to the space station over the 15 min interval.

Problem 3

Curtis 7.8

Spacecraft A and B are in the same circular earth orbit with a period of 2 h. B is 6 km ahead of A. At $t = 0$, B applies an in-track Δv (retrofire) of $3\,\mathrm{m\,s^{-1}}$. Using a CW frame attached to A, determine the distance between A and B at $t = 30\,\mathrm{min}$ and the velocity of B relative to A at that instant.

In the CW frame attached to A, the mean motion and elapsed angle are:

$$n = \frac{2\pi}{T} = \frac{2\pi}{7200\,\mathrm{s}} = 0.000\,872\,664\,6\,\mathrm{rad\,s^{-1}},$$
$$nt = \frac{2\pi}{2\,\mathrm{h}} \cdot 30\,\mathrm{min} = \frac{\pi}{2}.$$

Since A and B share the same circular orbit, B is stationary in A's CW frame before the burn: $\delta\mathbf{r}_0 = 6\hat{\mathbf{j}}$ (km) and $\delta\mathbf{v}_0^- = \mathbf{0}$. Indeed, with $\delta x_0 = 0$ and $\delta\mathbf{v}_0 = \mathbf{0}$, the CW solution gives $\delta\mathbf{r}(t) = \delta\mathbf{r}_0$ for all t.

The retrofire acts opposite to the direction of motion ($-\hat{\mathbf{j}}$), so immediately after the burn:

$$\delta\mathbf{v}_0^+ = \delta\mathbf{v}_0^- + \Delta\mathbf{v} = -3\hat{\mathbf{j}}\ (\mathrm{m\,s^{-1}}) = -0.003\hat{\mathbf{j}}\ (\mathrm{km\,s^{-1}}).$$

With $\delta x_0 = 0$ and $\delta\dot{x}_0^+ = 0$, only the second columns of the state transition matrices contribute. The second column of Φ_{vr} is zero, so δy_0 does not affect the relative velocity. At $nt = \frac{\pi}{2}$, where $\sin(nt) = 1$ and $\cos(nt) = 0$:

$$\delta x = \frac{2(1-\cos(nt))}{n}\delta\dot{y}_0^+ = (2291.831\,181\,\mathrm{s})(-0.003\,\mathrm{km\,s^{-1}}) = -6.875\,494\,\mathrm{km},$$
$$\delta y = \delta y_0 + \frac{4\sin(nt) - 3nt}{n}\delta\dot{y}_0^+$$
$$= 6\,\mathrm{km} + (-816.337\,639\,\mathrm{s})(-0.003\,\mathrm{km\,s^{-1}}) = 8.449\,013\,\mathrm{km},$$
$$\delta\dot{x} = 2\sin(nt)\delta\dot{y}_0^+ = 2(-3\,\mathrm{m\,s^{-1}}) = -6\,\mathrm{m\,s^{-1}},$$
$$\delta\dot{y} = (4\cos(nt) - 3)\delta\dot{y}_0^+ = -3(-3\,\mathrm{m\,s^{-1}}) = 9\,\mathrm{m\,s^{-1}},$$

with $\delta z = \delta\dot{z} = 0$.

Thus, the distance between A and B at $t = 30\,\mathrm{min}$ is:

$$d = |\delta\mathbf{r}| = \sqrt{\delta x^2 + \delta y^2} = 10.893\,036\,\mathrm{km} \quad \blacksquare$$

The velocity of B relative to A (measured in the CW frame) at that instant is:

$$\delta\mathbf{v} = -6\hat{\mathbf{i}} + 9\hat{\mathbf{j}}\ (\mathrm{m\,s^{-1}}),$$
$$|\delta\mathbf{v}| = \sqrt{6^2 + 9^2}\ \ \mathrm{m\,s^{-1}} = 10.816\,654\,\mathrm{m\,s^{-1}} \quad \blacksquare$$

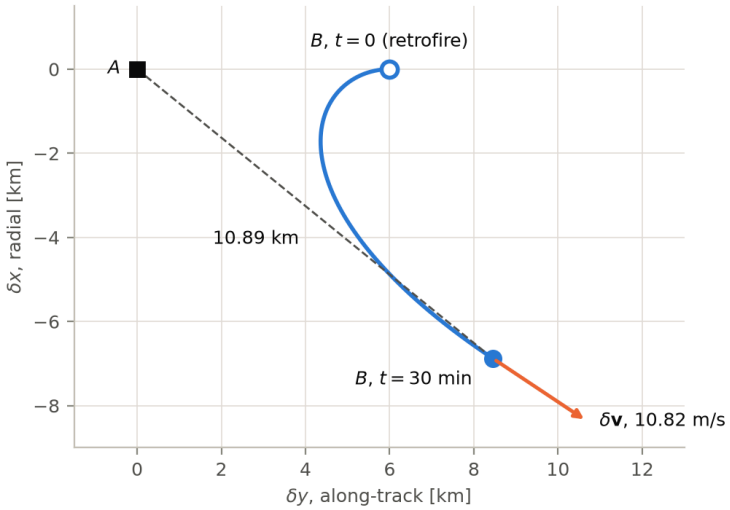


Figure 3: Motion of B relative to A after the retrofire, with the relative velocity at $t = 30\,\text{min}$.

Problem 4

Curtis 7.10

At time $t = 0$, a particle is at the origin of a CW frame with a relative velocity $\delta\mathbf{v}_0 = v\hat{\mathbf{j}}$. What will be the relative speed of the particle after a time equal to one-half the orbital period of the CW frame?

The particle starts at the origin, so $\delta\mathbf{r}_0 = \mathbf{0}$ and only the Φ_{vv} term of the CW solution survives in the relative velocity:

$$\delta\mathbf{v}(t) = \Phi_{vv}(t)\delta\mathbf{v}_0 = \begin{bmatrix} \cos(nt) & 2\sin(nt) & 0 \\ -2\sin(nt) & 4\cos(nt) - 3 & 0 \\ 0 & 0 & \cos(nt) \end{bmatrix} \begin{bmatrix} 0 \\ v \\ 0 \end{bmatrix} = v \begin{bmatrix} 2\sin(nt) \\ 4\cos(nt) - 3 \\ 0 \end{bmatrix}.$$

One-half of the orbital period is $t = \frac{T}{2} = \frac{\pi}{n}$, so $nt = \pi$, where $\sin(\pi) = 0$ and $\cos(\pi) = -1$:

$$\delta\mathbf{v}\left(\frac{T}{2}\right) = v \begin{bmatrix} 2\sin(\pi) \\ 4\cos(\pi) - 3 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ -7v \\ 0 \end{bmatrix}.$$

Thus, the relative speed after one-half of the orbital period is:

$$\left|\delta\mathbf{v}\left(\frac{T}{2}\right)\right| = 7|v| \quad \blacksquare$$

i.e., seven times the initial relative speed, now directed along $-\hat{\mathbf{j}}$.

More generally, the relative speed is:

$$|\delta\mathbf{v}| = |v|\sqrt{4\sin^2(nt) + (4\cos(nt) - 3)^2} = |v|\sqrt{13 - 24\cos(nt) + 12\cos^2(nt)},$$

whose radicand has derivative $24\sin(nt)(1 - \cos(nt)) \geq 0$ on $0 \leq nt \leq \pi$. The speed therefore grows monotonically from $|v|$ to its maximum of $7|v|$ at the half period. At that instant, $\delta\mathbf{r} = \Phi_{rv}\delta\mathbf{v}_0 = (v/n)[4, -3\pi, 0]$.

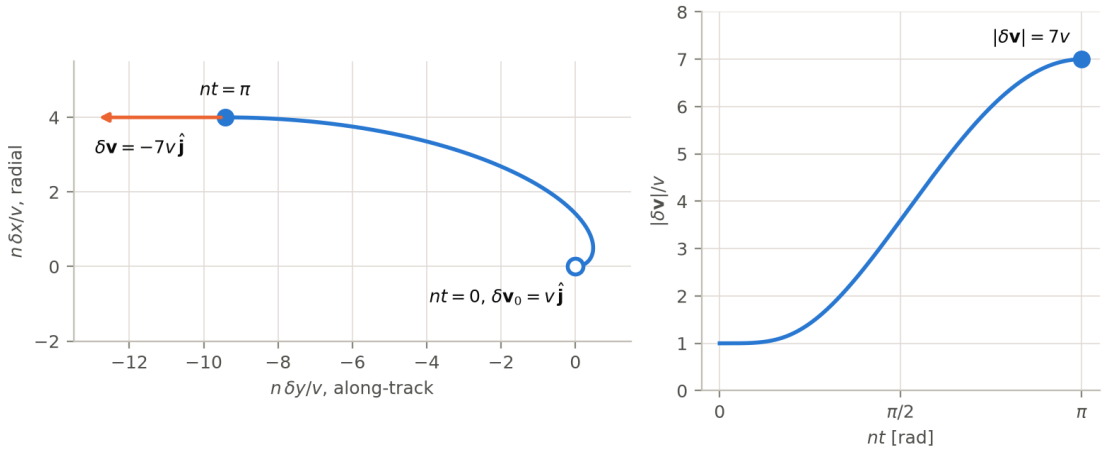


Figure 4: Nondimensional relative trajectory (left) and relative speed (right) over one-half of the orbital period.

Problem 5

Orbit Propagation with STM

a) Using Lagrange Coefficients, find the position, velocity, and time for this orbit 30 min after epoch:

$$\boldsymbol{r}_1 = \begin{bmatrix} 5740.13268349 \\ 3314.06715 \\ 0 \end{bmatrix} \quad (\mathrm{km})$$

$$\dot{\boldsymbol{r}}_1 = \begin{bmatrix} -2.75082684 \\ 4.76457184 \\ 5.5016536 \end{bmatrix} \quad (\mathrm{km\,s^{-1}})$$

b) Find the two-body state transition matrix over 30 min using Battin's technique for computing the two-body state transition matrix as described by Arora et. al, Section IV.A and in class.

c) Tellurion will propagate and compute the state transition matrix simultaneously. You should specify the two-body propagator, and refer to the STM tutorial for examples on how to specify it. Although Tellurion requires an epoch time, what you choose for the initial state will not matter because of the two-body propagation; if you use `tell.abstime(0.0)` it will use the current time.

Propagate using the two-body propagator in Tellurion and compute the state transition matrix. Compare with your results above and explain any differences you find.

a) As in class, write the Lagrange coefficients in terms of the change in eccentric anomaly ΔE. With $\mu = 398\,600\,\mathrm{km^3\,s^{-2}}$, the initial state gives:

$$r_1 = |\mathbf{r}_1| = 6628.134\,300\,\mathrm{km}, \quad v_1 = |\dot{\mathbf{r}}_1| = 7.780\,513\,\mathrm{km\,s^{-1}},$$

$$a = \left(\frac{2}{r_1} - \frac{v_1^2}{\mu}\right)^{-1} = 6672.381\,814\,\mathrm{km},$$

$$\sigma_1 = \frac{\mathbf{r}_1 \cdot \dot{\mathbf{r}}_1}{\sqrt{\mu}} = -0.0000000509 \ \sqrt{\mathrm{km}}.$$

The orbit is a nearly circular ellipse ($e = 0.0066314422$, $T = 5424.160\,805\,\mathrm{s}$), so 30 min is about one-third of a revolution.

Since $r = a(1 - e\cos E)$ and $\mathbf{r} \cdot \dot{\mathbf{r}} = \sqrt{\mu a}\, e \sin E$, the initial eccentric anomaly enters only through

$$e\cos(E_1) = 1 - \frac{r_1}{a}, \quad e\sin(E_1) = \frac{\sigma_1}{\sqrt{a}}.$$

Expanding $\sin(E_1 + \Delta E)$ in Kepler's equation, $n\Delta t = \Delta E - e(\sin(E_2) - \sin(E_1))$, gives the equation for ΔE:

$$n\Delta t = \Delta E - \left(1 - \frac{r_1}{a}\right)\sin(\Delta E) + \frac{\sigma_1}{\sqrt{a}}(1 - \cos(\Delta E)), \quad n = \sqrt{\frac{\mu}{a^3}}.$$

With $\Delta t = 1800\,\mathrm{s}$, $n\Delta t = 2.085\,066\,051\,6\,\mathrm{rad}$. Newton's method (derivative $r/a > 0$) converges to:

$$\Delta E = 2.090\,820\,864\,4\,\mathrm{rad} = 119.795\,211°.$$

The radius at t_2 and the Lagrange coefficients are then:

$$
\begin{aligned}
r_2 &= a + (r_1 - a)\cos(\Delta E) + \sigma_1\sqrt{a}\sin(\Delta E) = 6694.368\,463\,\mathrm{km},\\
f &= 1 - \frac{a}{r_1}(1 - \cos(\Delta E)) = -0.5068943144,\\
g &= \Delta t - \sqrt{\frac{a^3}{\mu}}(\Delta E - \sin(\Delta E)) = 744.194\,129\,\mathrm{s},\\
\dot f &= \frac{-\sqrt{\mu a}\sin(\Delta E)}{r_2 r_1} = -0.001\,008\,628\,7\ \mathrm{s^{-1}},\\
\dot g &= 1 - \frac{a}{r_2}(1 - \cos(\Delta E)) = -0.4919850836.
\end{aligned}
$$

Thus, 30 min after epoch, the position, velocity, and time are:

$$
\begin{aligned}
\mathbf{r}_2 &= f\mathbf{r}_1 + g\dot{\mathbf{r}}_1 = [-4956.789805, 1865.884594, 4094.298308]\ (\mathrm{km}),\\
\dot{\mathbf{r}}_2 &= \dot f\mathbf{r}_1 + \dot g\dot{\mathbf{r}}_1 = [-4.436296806, -5.686761523, -2.706731506]\ (\mathrm{km\,s^{-1}}),\\
t_2 &= t_1 + 1800\,\mathrm{s}\quad\blacksquare
\end{aligned}
$$

With the epoch used in part (c), $t_1 =$ 2026-10-21 00:00:00 UTC, this is $t_2 =$ 2026-10-21 00:30:00 UTC. As a check, $f\dot g - \dot f g = 1.000000000000$, $\mathbf{h}$ and $\mathcal{E}$ are conserved, and an RK4 integration of $\ddot{\mathbf{r}} = -\mu\mathbf{r}/r^3$ reproduces $\mathbf{r}_2$ to within 1 mm.

b) Battin's state transition matrix (Arora et al., Section IV.A) gives the Jacobian of the final state with respect to the initial state in four 3×3 blocks:

$$
\Phi(t_2, t_1) = \begin{bmatrix} A & B \\ C & D \end{bmatrix} = \begin{bmatrix} \frac{\partial \mathbf{r}_2}{\partial \mathbf{r}_1} & \frac{\partial \mathbf{r}_2}{\partial \dot{\mathbf{r}}_1} \\ \frac{\partial \dot{\mathbf{r}}_2}{\partial \mathbf{r}_1} & \frac{\partial \dot{\mathbf{r}}_2}{\partial \dot{\mathbf{r}}_1} \end{bmatrix}.
$$

The blocks use the universal functions U_1–U_5, which follow from the Lagrange coefficients of part (a) (Eqs. 38–44, 50):

$$
\begin{aligned}
\alpha &= \frac{2}{r_1} - \frac{v_1^2}{\mu},\quad \sigma_i = \frac{\mathbf{r}_i \cdot \dot{\mathbf{r}}_i}{\sqrt{\mu}},\quad \chi = \alpha\sqrt{\mu}\Delta t + \sigma_2 - \sigma_1,\\
U_1 &= -\frac{r_2 r_1 \dot f}{\sqrt{\mu}},\quad U_2 = r_1(1 - f),\quad U_3 = \sqrt{\mu}(\Delta t - g),\\
U_4 &= U_1 U_3 - \frac{1}{2}(U_2^2 - \alpha U_3^2),\quad U_5 = \frac{\chi^3/6 - U_3}{\alpha},\\
\overline{C} &= \frac{1}{\sqrt{\mu}}(3U_5 - \chi U_4 - \sqrt{\mu}\Delta t U_2).
\end{aligned}
$$

Substituting:

$$
\begin{aligned}
\alpha &= 0.000\,149\,871\,519\ \mathrm{km^{-1}},\quad \sigma_2 = 0.470\,079\,807\,8\ \sqrt{\mathrm{km}},\quad \chi = 170.787\,967\,555\,6\ \sqrt{\mathrm{km}},\\
U_1 &= 70.886\,509\ \sqrt{\mathrm{km}},\quad U_2 = 9987.897\,891\,\mathrm{km},\quad U_3 = 666\,580.676\,696\ \mathrm{km^{3/2}},\\
U_4 &= 30\,668\,715.844\,380\,\mathrm{km^2},\quad U_5 = 1\,092\,213\,156.875\,638\ \mathrm{km^{5/2}},\\
\overline{C} &= -21\,084\,599.541\,402\,\mathrm{km\,s}.
\end{aligned}
$$

For an ellipse, $\chi = \sqrt{a}\Delta E$, which matches the value above. With $\mathbf{r}_i$, $\dot{\mathbf{r}}_i$ the initial and final states and I the identity, the blocks are (Eqs. 46–49):

$$A = \frac{r_2}{\mu}(\dot{\mathbf{r}}_2 - \dot{\mathbf{r}}_1)(\dot{\mathbf{r}}_2 - \dot{\mathbf{r}}_1)^\mathsf{T} + \frac{1}{r_1^3}\left[r_1(1-f)\mathbf{r}_2\mathbf{r}_1^\mathsf{T} + \overline{C}\dot{\mathbf{r}}_2\mathbf{r}_1^\mathsf{T}\right] + fI,$$

$$B = \frac{r_1}{\mu}(1-f)\left[(\mathbf{r}_2 - \mathbf{r}_1)\dot{\mathbf{r}}_1^\mathsf{T} - (\dot{\mathbf{r}}_2 - \dot{\mathbf{r}}_1)\mathbf{r}_1^\mathsf{T}\right] + \frac{\overline{C}}{\mu}\dot{\mathbf{r}}_2\dot{\mathbf{r}}_1^\mathsf{T} + gI,$$

$$C = -\frac{1}{r_1^2}(\dot{\mathbf{r}}_2 - \dot{\mathbf{r}}_1)\mathbf{r}_1^\mathsf{T} - \frac{1}{r_2^2}\mathbf{r}_2(\dot{\mathbf{r}}_2 - \dot{\mathbf{r}}_1)^\mathsf{T} - \frac{\mu\overline{C}}{r_1^3 r_2^3}\mathbf{r}_2\mathbf{r}_1^\mathsf{T}$$
$$+\dot{f}\left[I - \frac{1}{r_2^2}\mathbf{r}_2\mathbf{r}_2^\mathsf{T} + \frac{1}{\mu r_2}(\mathbf{r}_2\dot{\mathbf{r}}_2^\mathsf{T} - \dot{\mathbf{r}}_2\mathbf{r}_2^\mathsf{T})\mathbf{r}_2(\dot{\mathbf{r}}_2 - \dot{\mathbf{r}}_1)^\mathsf{T}\right],$$

$$D = \frac{r_1}{\mu}(\dot{\mathbf{r}}_2 - \dot{\mathbf{r}}_1)(\dot{\mathbf{r}}_2 - \dot{\mathbf{r}}_1)^\mathsf{T} + \frac{1}{r_2^3}\left[r_1(1-f)\mathbf{r}_2\mathbf{r}_1^\mathsf{T} - \overline{C}\mathbf{r}_2\dot{\mathbf{r}}_1^\mathsf{T}\right] + \dot{g}I.$$

Thus, the two-body state transition matrix over 30 min is $\Phi = \left[\begin{smallmatrix} A & B \\ C & D \end{smallmatrix}\right]$ with:

$$A = \begin{bmatrix} 0.4087630 & 0.7969548 & 0.2323545 \\ 3.0268423 & 2.9043390 & 1.4407935 \\ 2.1634937 & 2.5557372 & 0.6246922 \end{bmatrix}, \qquad B = \begin{bmatrix} 1078.4221 & -19.0402 & -183.6032 \\ 775.5909 & 2872.4323 & 1455.3128 \\ 504.5674 & 1852.6284 & 2096.3336 \end{bmatrix} \text{ (s)},$$

$$C = \begin{bmatrix} -3.0334216 & -2.0377852 & -0.7523773 \\ 2.4206950 & 1.8971680 & 1.3061462 \\ 3.1087798 & 3.5353863 & 0.4987157 \end{bmatrix} \times 10^{-3}\ (\mathrm{s}^{-1}), \quad D = \begin{bmatrix} -0.4337060 & -1.9138001 & -1.6865391 \\ 0.2887624 & 2.1550312 & 2.1480021 \\ 0.2209360 & 3.2492833 & 2.2115088 \end{bmatrix} \quad \blacksquare$$

As checks, central differences of the part (a) propagation (1 m and 1 mm s^{-1} steps) match Φ to a relative $6.7 \cdot 10^{-10}$. Φ is also symplectic, $\Phi^\mathsf{T} J \Phi = J$ to $1.4 \cdot 10^{-12}$ with $J = \left[\begin{smallmatrix} 0 & I \\ -I & 0 \end{smallmatrix}\right]$, so $\det(\Phi) = 1.000000000000$.

c)

./src/s05c.py — Python

```python
import numpy as np
import astropy.units as u
import tellurion as tell

r1 = np.array([5740.13268349, 3314.06715, 0.0])
v1 = np.array([-2.75082684, 4.76457184, 5.5016536])

# Two-body (Keplerian) propagation with the state transition matrix. The epoch
# is fixed for reproducible output; it does not affect two-body motion.
epoch = tell.abstime("2026-10-21T00:00:00")
state1 = tell.pvtcart(np.hstack((r1, v1)), epoch)
events = {"altitude": 125.0 * u.km, "eclipse": [], "visibility": [], "stm": True}
gen = tell.prepare(state1, 30 * u.min, events, propagator="keplerian")

state2 = tell.propagate(gen, 30 * u.min, include_init=False, output="pvt")
r2 = state2.position_vector.to_value(u.km)
v2 = state2.velocity_vector.to_value(u.km / u.s)
stm = gen["final"]["stm"]
mu = tell.force.deffe["earthmu"].to_value(u.km**3 / u.s**2)

np.set_printoptions(precision=10, suppress=False, linewidth=100)
print(f"mu = {mu} km^3/s^2")
```

```python
print(f"t1 = {epoch.isot}")
print(f"t2 = {state2.time.isot}")
print(f"r2 = {r2} km")
print(f"v2 = {v2} km/s")
for name, block in (
    ("A = dr2/dr1 [-]", stm[:3, :3]),
    ("B = dr2/dv1 [s]", stm[:3, 3:]),
    ("C = dv2/dr1 [1/s]", stm[3:, :3]),
    ("D = dv2/dv1 [-]", stm[3:, 3:]),
):
    print(f"{name} =\n{block}")
```

```
./outputs/text/s05c.txt
mu = 398600.4415 km^3/s^2
t1 = 2026-10-21T00:00:00.000
t2 = 2026-10-21T00:30:00.000
r2 = [-4956.7919746134  1865.8680932268  4094.2851028721] km
v2 = [-4.4362831699 -5.6867789353 -2.7067534035] km/s
A = dr2/dr1 [-] =
[[0.4087563301 0.7969496852 0.2323525836]
 [3.0268476722 2.9043431511 1.4407957721]
 [2.163500863  2.5557447511 0.6246935912]]
B = dr2/dv1 [s] =
[[1078.4194479575  -19.0443942669 -183.6067865125]
 [ 775.5914514231 2872.4343089366 1455.3163953159]
 [ 504.568033657  1852.6343129906 2096.3360731709]]
C = dv2/dr1 [1/s] =
[[-0.0030334438 -0.0020378067 -0.0007523846]
 [ 0.0024206948  0.0018971673  0.0013061453]
 [ 0.003108791   0.0035353961  0.0004987191]]
D = dv2/dv1 [-] =
[[-0.4337101817 -1.9138173757 -1.6865538847]
 [ 0.2887600387  2.1550291783  2.1480049945]
 [ 0.2209340852  3.2492936874  2.2115147362]]
```

The script propagates the same initial state with Tellurion's Keplerian (two-body) propagator and requests the state transition matrix with `"stm": True`. Compared with parts (a) and (b):

$$\mathbf{r}_2^{\mathrm{Tell}} - \mathbf{r}_2 = [-2.1698, -16.5010, -13.2054]\ (\mathrm{m}), \quad |\Delta\mathbf{r}_2| = 21.2455\,\mathrm{m},$$
$$\left|\dot{\mathbf{r}}_2^{\mathrm{Tell}} - \dot{\mathbf{r}}_2\right| = 31.1226\,\mathrm{mm\,s^{-1}}.$$

The largest entry of each block of $\Phi^{\mathrm{Tell}} - \Phi$, relative to that block, is $2.5 \cdot 10^{-6}$ (A), $2.1 \cdot 10^{-6}$ (B), $6.3 \cdot 10^{-6}$ (C), and $5.3 \cdot 10^{-6}$ (D).

These differences come from the gravitational parameter. Tellurion takes μ from Orekit's gravity field, $\mu^{\mathrm{Tell}} = 398\,600.4415\,\mathrm{km^3\,s^{-2}}$, while parts (a) and (b) used $\mu = 398\,600\,\mathrm{km^3\,s^{-2}}$. The difference is $\Delta\mu = 0.4415\,\mathrm{km^3\,s^{-2}}$, or $1.11 \cdot 10^{-6}$ relative. Linearizing the propagation in μ predicts:

$$\frac{\partial \mathbf{r}_2}{\partial \mu}\Delta\mu = [-2.1698, -16.5010, -13.2054]\ (\mathrm{m}),$$

which reproduces the observed position difference. Repeating parts (a) and (b) with μ^{Tell} leaves only round-off: $|\Delta\mathbf{r}_2| = 1.8 \cdot 10^{-9}\,\mathrm{m}$, $|\Delta\dot{\mathbf{r}}_2| = 2.0 \cdot 10^{-9}\,\mathrm{mm\,s^{-1}}$, and the STM agrees to $1.0 \cdot 10^{-14}$.

Nothing else contributes:

- The epoch is irrelevant to two-body motion.
- Orekit works in SI units, but the STM blocks have units of $[-, \mathrm{s}; \mathrm{s}^{-1}, -]$, which are the same in kilometers.
- Orekit's analytic Keplerian propagator and Battin's formulation agree to machine precision once μ matches.

Thus, Tellurion agrees with parts (a) and (b) to within the effect of its slightly larger μ:

$$|\Delta\mathbf{r}_2| = 21.2455\,\mathrm{m}, \quad |\Delta\dot{\mathbf{r}}_2| = 31.1226\,\mathrm{mm\,s^{-1}}, \quad \frac{\max_{\mathrm{block}}|\Delta\Phi_{ij}|}{\max_{\mathrm{block}}|\Phi_{ij}|} \le 6.3 \cdot 10^{-6} \quad \blacksquare$$

Problem 6

Lambert Problem

Solve the Lambert, or two-point boundary value problem for the zero-revolution case with these terminal points and a time of flight of 5200 s.

$$\boldsymbol{r}_1 = \begin{bmatrix} -6991 \\ -5118 \\ 7078 \end{bmatrix} \quad \text{(km)}$$

$$\boldsymbol{r}_2 = \begin{bmatrix} 11326 \\ -6317 \\ -6677 \end{bmatrix} \quad \text{(km)}$$

a) Using the method shown in class, compute the initial and final velocities.

b) Find the minimum geocentric distance over each orbit segment, and whether it is sufficiently large to avoid crashing into the earth.

c) Recompute Part A with Tellurion. Do your results agree?

d) Propagate the initial state to the final time and compare the resultant position vector with the final position vector $\boldsymbol{r}_2$ given in the problem. You may use Tellurion for this task; please use an analytic two-body propagator.

Tellurion solves the Lambert problem directly from two position inputs. Positions are made with the function `tell.pvtcart()`, use `tell.lambert()` to solve this problem.

a) Following the method from class (Prussing and Conway), Lambert's theorem makes the flight time a function of only the semimajor axis a, the radius sum $r_1 + r_2$, the chord c, and the direction (short or long way). With $\mu = 398\,600\,\text{km}^3\,\text{s}^{-2}$:

$$r_1 = 11187.765\,148\,\text{km}, \quad r_2 = 14586.469\,552\,\text{km},$$

$$c = |\mathbf{r}_2 - \mathbf{r}_1| = 22937.962\,311\,\text{km}, \quad s = \frac{r_1 + r_2 + c}{2} = 24356.098\,506\,\text{km},$$

$$\cos(\Delta\theta) = \frac{\mathbf{r}_1 \cdot \mathbf{r}_2}{r_1 r_2} = -0.5766864911.$$

The zero-revolution problem has two solutions. The short way has $\Delta\theta = 125.217\,824° < 180°$, and since $(\mathbf{r}_1 \times \mathbf{r}_2) \cdot \hat{\mathbf{K}} > 0$ it is prograde. The long way has $\Delta\theta = 234.782\,176°$ and travels in the opposite sense.

For a trial a, the angles α and β follow from

$$\sin^2\left(\frac{\alpha^*}{2}\right) = \frac{s}{2a}, \quad \sin^2\left(\frac{\beta^*}{2}\right) = \frac{s-c}{2a}, \quad 0 \le \alpha^*, \beta^* \le \pi,$$

with $\beta = \beta^*$ on the short way and $\beta = -\beta^*$ on the long way. The flight time is Lagrange's equation:

$$\Delta t(a) = \frac{1}{n}[\alpha - \beta - (\sin(\alpha) - \sin(\beta))], \quad n = \sqrt{\frac{\mu}{a^3}}.$$

Two limits fix the branch:

- The minimum-energy ellipse, $a_{\min} = s/2 = 12178.049\,253\,\text{km}$ with $\alpha = \pi$, has flight time $t_c = [\pi - (\beta_c - \sin(\beta_c))]/n_c$, where $\sin^2(\beta_c/2) = (s-c)/s$. If $\Delta t < t_c$ then $\alpha = \alpha^*$; otherwise $\alpha = 2\pi - \alpha^*$.

- The parabolic (Euler) time $t_p = \left[(r_1 + r_2 + c)^{3/2} \mp (r_1 + r_2 - c)^{3/2}\right]/(6\sqrt{\mu})$ bounds the ellipses, with the upper sign for the short way and the lower sign for the long way: $\Delta t > t_p$ is required.

Starting from $a_{\min}$, bracketing and bisecting on a until the flight-time equation matches $\Delta t = 5200\,\mathrm{s}$ gives $\Delta E = \alpha - \beta$. The class ΔE Lagrange coefficients then give the velocities:

$$f = 1 - \frac{a}{r_1}(1 - \cos(\Delta E)), \quad g = \Delta t - \sqrt{\frac{a^3}{\mu}}(\Delta E - \sin(\Delta E)), \quad \dot{g} = 1 - \frac{a}{r_2}(1 - \cos(\Delta E)),$$

$$\dot{\mathbf{r}}_1 = \frac{\mathbf{r}_2 - f\mathbf{r}_1}{g}, \quad \dot{\mathbf{r}}_2 = \frac{\dot{g}\mathbf{r}_2 - \mathbf{r}_1}{g}.$$

	Short way	Long way
t_p	2798.283 597 s	2878.033 838 s
β_c	27.926 415°	−27.926 415°
t_c	6646.659 710 s	6727.848 154 s
a	12756.227 403 km	12846.463 795 km
α	155.415 634°	153.629 165°
β	27.273 667°	−27.175 867°
ΔE	128.141 967°	180.805 031°
f	−0.8443922996	−1.2964067648
g	1891.069 556 s	−2110.110 806 s
$\dot{g}$	−0.4146416865	−0.7613350150

Table 1: Lambert iteration on a for both zero-revolution transfers.

In both directions $t_p < 5200\,\mathrm{s} < t_c$, so both transfers are ellipses on the $\alpha = \alpha^*$ branch. The bisection converges in 47 iterations to a flight-time residual below $3.4 \cdot 10^{-11}\,\mathrm{s}$.

Thus, the initial and final velocities are:

$$\begin{aligned}
\text{Short way:}\quad & \dot{\mathbf{r}}_1 = [2.867611832, -5.625705176, -0.370367817]\ (\mathrm{km\,s^{-1}}),\\
& \dot{\mathbf{r}}_2 = [1.213476390, 4.091489660, -2.278836041]\ (\mathrm{km\,s^{-1}}),\\
\text{Long way:}\quad & \dot{\mathbf{r}}_1 = [-1.072370371, 6.138070941, -1.184282396]\ (\mathrm{km\,s^{-1}}),\\
& \dot{\mathbf{r}}_2 = [0.773362411, -4.704659708, 0.945242353]\ (\mathrm{km\,s^{-1}}) \quad \blacksquare
\end{aligned}$$

As a check, propagating each $(\mathbf{r}_1, \dot{\mathbf{r}}_1)$ for 5200 s with the Lagrange coefficients of Problem 5, Part A (and with RK4) returns $\mathbf{r}_2$ to within 1 mm.

b) The distance from the focus decreases toward perigee and increases after it. Its minimum over a transfer arc is therefore the perigee radius $r_p = a(1 - e)$ if the arc passes through perigee, and otherwise the nearer endpoint, $\min(r_1, r_2)$.

For each transfer, $\mathbf{e} = (\dot{\mathbf{r}}_1 \times \mathbf{h})/\mu - \mathbf{r}_1/r_1$ and $\mathbf{h} = \mathbf{r}_1 \times \dot{\mathbf{r}}_1$ give e and the true anomaly θ_1 at $\mathbf{r}_1$. The arc then spans $\theta_1 \le \theta \le \theta_1 + \Delta\theta$, and it passes perigee exactly when $\theta_1 + \Delta\theta \ge 360°$.

$$\begin{aligned}
\text{Short way:}\quad & e = 0.1499758597, \quad r_p = 10843.101\,231\,\mathrm{km}, \quad \theta_1 = 40.201\,576°, \quad \theta_1 + \Delta\theta = 165.419\,400°,\\
\text{Long way:}\quad & e = 0.4694844331, \quad r_p = 6815.249\,022\,\mathrm{km}, \quad \theta_1 = 257.097\,251°, \quad \theta_1 + \Delta\theta = 491.879\,427°.
\end{aligned}$$

The short-way arc moves away from perigee the whole time, so its closest point is the start, $\mathbf{r}_1$. The long-way arc crosses perigee at $t = 1939.111\,201\,\mathrm{s}$ after departure. With $R_\oplus = 6378\,\mathrm{km}$, the minimum geocentric distances are:

$$\begin{aligned}
\text{Short way:}\quad & r_{\min} = r_1 = 11187.765\,148\,\mathrm{km}, \quad h_{\min} = r_{\min} - R_\oplus = 4809.765\,148\,\mathrm{km}, \\
\text{Long way:}\quad & r_{\min} = r_p = 6815.249\,022\,\mathrm{km}, \quad h_{\min} = r_{\min} - R_\oplus = 437.249\,022\,\mathrm{km}, \\
\Rightarrow\quad & \text{neither transfer crashes into the Earth} \quad \blacksquare
\end{aligned}$$

Both minima are well above Earth's surface. The long way's 437 km perigee is also above the sensible atmosphere (roughly 100 km to 125 km, the default altitude stop in Tellurion), although the short way stays far higher. Sampling each arc at 2001 points gives the same minima to within 1 km.

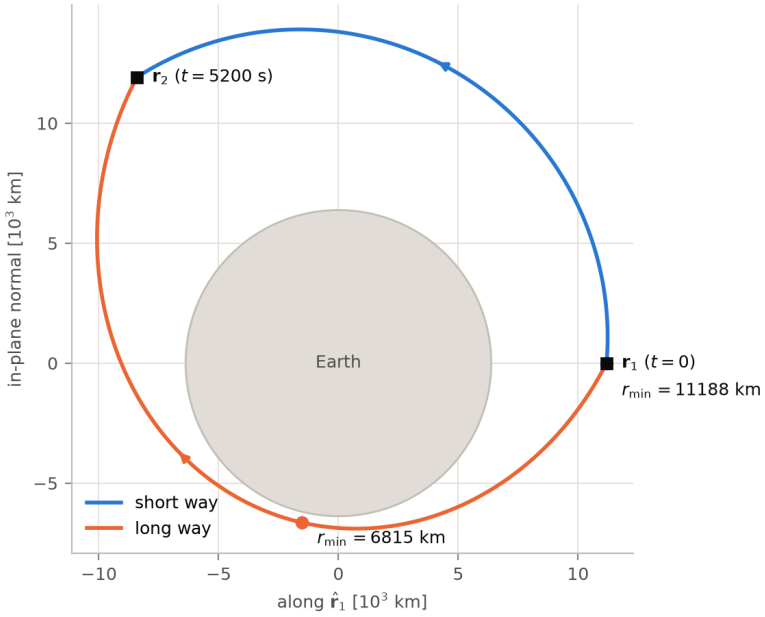


Figure 5: Both zero-revolution transfers from $\mathbf{r}_1$ to $\mathbf{r}_2$ in the transfer plane, with each arc's closest approach.

c)

./src/s06c.py

```python
import numpy as np
import astropy.units as u
import tellurion as tell

# Fixed epoch for reproducible output; only the 5200 s separation matters.
now = tell.abstime("2026-10-21T00:00:00")
r1 = tell.pvtcart(np.array([-6991, -5118, 7078]), now)
print(r1.cartesian)

r2 = tell.pvtcart(np.array([11326, -6317, -6677]), now + 5200 * u.s)
print(r2.cartesian)

# Zero-revolution Lambert solutions in both directions.
mu = tell.force.deffe["earthmu"].to_value(u.km**3 / u.s**2)
velocities = {}
```

```python
for shortway in (True, False):
    branch = "short" if shortway else "long"
    solution = tell.lambert(r1, r2, shortway=shortway)
    v1 = solution[0].velocity_vector.to_value(u.km / u.s)
    v2 = solution[1].velocity_vector.to_value(u.km / u.s)
    velocities[branch] = (v1, v2)

np.set_printoptions(precision=10, suppress=False, linewidth=100)
print(f"mu = {mu} km^3/s^2")
for branch, (v1, v2) in velocities.items():
    print(f"{branch} way:")
    print(f"  v1 = {v1} km/s")
    print(f"  v2 = {v2} km/s")
```

./outputs/text/s06c.txt

```
([-6991.0, -5118.0, 7078.0],) (km,)
([11326.0, -6317.0, -6677.0],) (km,)
mu = 398600.4415 km^3/s^2
short way:
  v1 = [ 2.8676112397 -5.6257088676 -0.3703661493] km/s
  v2 = [ 1.2134746633  4.0914926338 -2.2788356818] km/s
long way:
  v1 = [-1.0723731793  6.1380736599 -1.1842811179] km/s
  v2 = [ 0.7733603732 -4.7046615191  0.9452445211] km/s
```

`tell.lambert()` (Orekit's `IodLambert`) returns the short-way (`shortway=True`) and long-way (`shortway=False`) solutions. The differences from part (a) are:

$$\text{Short way:}\quad |\Delta\dot{\mathbf{r}}_1| = 4.0937\,\text{mm}\,\text{s}^{-1},\quad |\Delta\dot{\mathbf{r}}_2| = 3.4577\,\text{mm}\,\text{s}^{-1},$$
$$\text{Long way:}\quad |\Delta\dot{\mathbf{r}}_1| = 4.1129\,\text{mm}\,\text{s}^{-1},\quad |\Delta\dot{\mathbf{r}}_2| = 3.4830\,\text{mm}\,\text{s}^{-1}.$$

These differences are about one part in 10^6, the same size as the gravitational-parameter difference found in Problem 5, Part C: Tellurion uses $\mu^{\text{Tell}} = 398\,600.4415\,\text{km}^3\,\text{s}^{-2}$. Re-solving with the class method and μ^{Tell} shifts a by only $-0.003\,027\,\text{km}$ (short) and $-0.003\,386\,\text{km}$ (long). Both velocities on both ways then agree with Tellurion to within $9.3 \cdot 10^{-8}\,\text{mm}\,\text{s}^{-1}$.

Thus, yes, the results agree to within a few $\text{mm}\,\text{s}^{-1}$ (about one part in 10^6) for both transfers, and to round-off once the same μ is used:

$$\max(|\Delta\dot{\mathbf{r}}_1|, |\Delta\dot{\mathbf{r}}_2|) \le 4.1129\,\text{mm}\,\text{s}^{-1} \quad \text{(short and long way)} \quad \blacksquare$$

d)

./src/s06d.py Python

```python
import numpy as np
import astropy.units as u
import tellurion as tell

# Fixed epoch for reproducible output; only the 5200 s separation matters.
now = tell.abstime("2026-10-21T00:00:00")
r1 = tell.pvtcart(np.array([-6991, -5118, 7078]), now)
```

```python
print(r1.cartesian)

r2 = tell.pvtcart(np.array([11326, -6317, -6677]), now + 5200 * u.s)
print(r2.cartesian)

# Propagate each Lambert initial state to the final time with the analytic
# two-body propagator, then compare with the given final position.
tof = 5200 * u.s
np.set_printoptions(precision=10, suppress=False, linewidth=100)
misses = {}
for shortway in (True, False):
    branch = "short" if shortway else "long"
    initial = tell.lambert(r1, r2, shortway=shortway)[0]
    gen = tell.prepare(initial, tof, propagator="keplerian")
    final = tell.propagate(gen, tof, include_init=False, output="pvt")
    miss = (final.position_vector - r2.position_vector).to(u.m)
    misses[branch] = np.linalg.norm(miss.value)
    print(f"{branch} way:")
    print(f"  t2 = {final.time.isot}")
    print(f"  r(t2) = {final.position_vector.to_value(u.km)} km")
    print(f"  r(t2) - r2 = {miss.value} m")
    print(f"  |r(t2) - r2| = {misses[branch]:.3e} m")
```

```
./outputs/text/s06d.txt
([-6991.0, -5118.0, 7078.0],) (km,)
([11326.0, -6317.0, -6677.0],) (km,)
short way:
  t2 = 2026-10-21T01:26:40.000
  r(t2) = [11326. -6317. -6677.] km
  r(t2) - r2 = [ 0.0000000000e+00  4.5474735089e-09 -2.7284841053e-09] m
  |r(t2) - r2| = 5.303e-09 m
long way:
  t2 = 2026-10-21T01:26:40.000
  r(t2) = [11326. -6317. -6677.] km
  r(t2) - r2 = [ 7.2759576142e-09  1.1823431123e-08 -8.1854523160e-09] m
  |r(t2) - r2| = 1.612e-08 m
```

The script takes each Lambert initial state from `tell.lambert()`, propagates it with Tellurion's analytic two-body (`"keplerian"`) propagator for 5200 s, and compares the result with the given $\mathbf{r}_2$:

$$\text{Short way:}\quad |\mathbf{r}(t_2)-\mathbf{r}_2| = 5.3\cdot10^{-9}\,\text{m},$$
$$\text{Long way:}\quad |\mathbf{r}(t_2)-\mathbf{r}_2| = 1.6\cdot10^{-8}\,\text{m}\quad\blacksquare$$

Both transfers land on $\mathbf{r}_2$ to within double-precision round-off, so the Lambert solutions satisfy the boundary conditions. This depends on using a consistent μ. Flying the part (a) velocities ($\mu = 398\,600\,\text{km}^3\,\text{s}^{-2}$) with Tellurion's μ^{Tell} would miss $\mathbf{r}_2$ by 51.9363 m (short) and 63.0586 m (long).

Code

./src/index.py Python

```python
import contextlib
import io
import runpy
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np

MU_EARTH = 398600.0  # km^3/s^2
R_EARTH = 6378.0  # km
TAU = 2.0 * np.pi
ROOT = Path(__file__).resolve().parents[1]
TEXT_DIR = ROOT / "outputs" / "text"
FIGURE_DIR = ROOT / "outputs" / "figures"

# Figure ink and categorical series colors (fixed order).
INK = "#0b0b0b"
INK_SECONDARY = "#52514e"
MUTED = "#898781"
GRID = "#e1e0d9"
BASELINE = "#c3c2b7"
SERIES = ("#2a78d6", "#eb6834", "#1baf7a")

plt.rcParams.update(
    {
        "font.size": 9,
        "axes.edgecolor": BASELINE,
        "axes.labelcolor": INK_SECONDARY,
        "axes.spines.top": False,
        "axes.spines.right": False,
        "axes.grid": True,
        "axes.axisbelow": True,
        "grid.color": GRID,
        "grid.linewidth": 0.6,
        "xtick.color": MUTED,
        "ytick.color": MUTED,
        "xtick.labelcolor": INK_SECONDARY,
        "ytick.labelcolor": INK_SECONDARY,
        "legend.frameon": False,
        "savefig.dpi": 200,
    }
)


```

```
45  def heading(problem: str) -> None:
46      """Print the heading used to separate each problem's output."""
47      print(f"\n------\n {problem}\n------")
48
49
50  def clean(value: float) -> float:
51      """Zero out round-off so values like -1e-17 do not print as -0.000000."""
52      return 0.0 if abs(value) < 1e-12 else float(value)
53
54
55  def sci(value: float, digits: int = 1) -> str:
56      """Scientific notation without exponent padding, e.g. 2.5e-6 (for unify)."""
57      mantissa, exponent = f"{value:.{digits}e}".split("e")
58      return f"{mantissa}e{int(exponent)}"
59
60
61  def write_output(name: str, lines: list[str]) -> None:
62      """Print and save a solution's text output for inclusion in Typst."""
63      output = "\n".join(lines) + "\n"
64      print(output, end="")
65      TEXT_DIR.mkdir(parents=True, exist_ok=True)
66      (TEXT_DIR / f"s{name}.txt").write_text(output, encoding="utf-8")
67
68
69  def save_figure(fig: plt.Figure, name: str) -> None:
70      """Save a solution's figure for inclusion in Typst."""
71      FIGURE_DIR.mkdir(parents=True, exist_ok=True)
72      path = FIGURE_DIR / f"s{name}.png"
73      fig.savefig(path, bbox_inches="tight")
74      plt.close(fig)
75      print(f"saved {path.relative_to(ROOT)}")
76
77
78  def vector_lines(
79      name: str, vector: np.ndarray, unit: str = "", precision: int = 6
80  ) -> list[str]:
81      """Format each component of a vector as its own key/value line."""
82      return [
83          f"{name}_{axis} = {clean(value):.{precision}f} {unit}".rstrip()
84          for axis, value in zip("xyz", vector)
85      ]
86
87
88  def rk4(rate, state: np.ndarray, duration: float, steps: int) -> np.ndarray:
89      """Integrate state' = rate(state) with fixed-step fourth-order Runge-Kutta."""
90      dt = duration / steps
```

```
91        for _ in range(steps):
92            k1 = rate(state)
93            k2 = rate(state + dt / 2.0 * k1)
94            k3 = rate(state + dt / 2.0 * k2)
95            k4 = rate(state + dt * k3)
96            state = state + dt / 6.0 * (k1 + 2.0 * k2 + 2.0 * k3 + k4)
97        return state
98
99
100   def two_body_rate(state: np.ndarray) -> np.ndarray:
101       """Two-body equations of motion, r'' = -mu r / r^3."""
102       r, v = state[:3], state[3:]
103       return np.concatenate((v, -MU_EARTH * r / np.linalg.norm(r) ** 3))
104
105
106   def cw_rate(n: float):
107       """Linearized Clohessy-Wiltshire equations (x radial, y along-track)."""
108
109       def rate(state: np.ndarray) -> np.ndarray:
110           x, _, z, vx, vy, vz = state
111           return np.array(
112               [vx, vy, vz, 3.0 * n * n * x + 2.0 * n * vy, -2.0 * n * vx, -n * n * z]
113           )
114
115       return rate
116
117
118   def solve_hyperbolic_kepler(e: float, mean_anomaly: float) -> float:
119       """Solve e sinh(F) - F = M_h for the hyperbolic eccentric anomaly F."""
120       anomaly = mean_anomaly
121       for _ in range(50):
122           step = (e * np.sinh(anomaly) - anomaly - mean_anomaly) / (
123               e * np.cosh(anomaly) - 1.0
124           )
125           anomaly -= step
126           if abs(step) < 1e-14:
127               return anomaly
128       raise RuntimeError("Hyperbolic Kepler iteration did not converge.")
129
130
131   def cw_stm(n: float, t: float) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
132       """Clohessy-Wiltshire state transition matrices (Curtis Eq. 7.53)."""
133       s, c, nt = np.sin(n * t), np.cos(n * t), n * t
134       phi_rr = np.array(
135           [[4.0 - 3.0 * c, 0.0, 0.0], [6.0 * (s - nt), 1.0, 0.0], [0.0, 0.0, c]]
136       )
```

```
137     phi_rv = np.array(
138         [
139             [s / n, 2.0 * (1.0 - c) / n, 0.0],
140             [2.0 * (c - 1.0) / n, (4.0 * s - 3.0 * nt) / n, 0.0],
141             [0.0, 0.0, s / n],
142         ]
143     )
144     phi_vr = np.array(
145         [[3.0 * n * s, 0.0, 0.0], [6.0 * n * (c - 1.0), 0.0, 0.0], [0.0, 0.0, -n * s]]
146     )
147     phi_vv = np.array(
148         [[c, 2.0 * s, 0.0], [-2.0 * s, 4.0 * c - 3.0, 0.0], [0.0, 0.0, c]]
149     )
150     return phi_rr, phi_rv, phi_vr, phi_vv
151
152
153 def cw_propagate(
154     n: float, t: float, dr0: np.ndarray, dv0: np.ndarray
155 ) -> tuple[np.ndarray, np.ndarray]:
156     """Relative position and velocity at time t from the CW STM."""
157     phi_rr, phi_rv, phi_vr, phi_vv = cw_stm(n, t)
158     return phi_rr @ dr0 + phi_rv @ dv0, phi_vr @ dr0 + phi_vv @ dv0
159
160
161 def check_cw(n: float, t: float, dr0: np.ndarray, dv0: np.ndarray) -> float:
162     """Compare the STM solution against direct integration of the CW equations."""
163     dr, dv = cw_propagate(n, t, dr0, dv0)
164     numerical = rk4(cw_rate(n), np.concatenate((dr0, dv0)), t, 6000)
165     error = np.linalg.norm(numerical - np.concatenate((dr, dv)))
166     assert error < 1e-9, error
167     return error
168
169
170 def matrix_lines(
171     name: str, matrix: np.ndarray, unit: str = "", precision: int = 6
172 ) -> list[str]:
173     """Format each entry of a matrix as its own key/value line (name_ij)."""
174     return [
175         f"{name}_{i + 1}{j + 1} = {clean(value):.{precision}f} {unit}".rstrip()
176         for i, row in enumerate(matrix)
177         for j, value in enumerate(row)
178     ]
179
180
181 def run_snippet(name: str) -> dict:
182     """Run a standalone Tellurion snippet, saving its printed output for Typst."""
```

```
    buffer = io.StringIO()
    # Snippets set their own numpy print options; restore ours afterwards.
    with np.printoptions(), contextlib.redirect_stdout(buffer):
        namespace = runpy.run_path(
            str(ROOT / "src" / f"{name}.py"), run_name="__main__"
        )
    output = buffer.getvalue()
    print(output, end="")
    TEXT_DIR.mkdir(parents=True, exist_ok=True)
    (TEXT_DIR / f"{name}.txt").write_text(output, encoding="utf-8")
    return namespace


def solve_kepler_delta_e(mean: float, ecos: float, esin: float) -> float:
    """Solve n dt = dE - (e cos E1) sin dE + (e sin E1)(1 - cos dE) for dE."""
    delta = mean
    for _ in range(50):
        step = (delta - ecos * np.sin(delta) + esin * (1.0 - np.cos(delta)) - mean) / (
            1.0 - ecos * np.cos(delta) + esin * np.sin(delta)
        )
        delta -= step
        if abs(step) < 1e-14:
            return delta
    raise RuntimeError("Elliptic Kepler iteration did not converge.")


def lagrange_propagate(
    r1_vec: np.ndarray, v1_vec: np.ndarray, dt: float, mu: float = MU_EARTH
) -> dict:
    """Two-body propagation with the Lagrange coefficients in Delta E."""
    r1 = np.linalg.norm(r1_vec)
    alpha = 2.0 / r1 - v1_vec @ v1_vec / mu
    if alpha <= 0.0:
        raise ValueError("The Delta E Lagrange coefficients require an ellipse.")
    a = 1.0 / alpha
    sigma1 = r1_vec @ v1_vec / np.sqrt(mu)
    # e cos(E1) and e sin(E1) from the initial state.
    ecos, esin = 1.0 - r1 / a, sigma1 / np.sqrt(a)
    mean = np.sqrt(mu / a**3) * dt
    delta = solve_kepler_delta_e(mean, ecos, esin)
    c, s = np.cos(delta), np.sin(delta)
    r = a + (r1 - a) * c + sigma1 * np.sqrt(a) * s
    f = 1.0 - a / r1 * (1.0 - c)
    g = dt - np.sqrt(a**3 / mu) * (delta - s)
    f_dot = -np.sqrt(mu * a) * s / (r * r1)
    g_dot = 1.0 - a / r * (1.0 - c)
```

```
    return {
        "a": a,
        "e": np.hypot(ecos, esin),
        "sigma1": sigma1,
        "ecos": ecos,
        "esin": esin,
        "M": mean,
        "dE": delta,
        "r": r,
        "f": f,
        "g": g,
        "f_dot": f_dot,
        "g_dot": g_dot,
        "r2": f * r1_vec + g * v1_vec,
        "v2": f_dot * r1_vec + g_dot * v1_vec,
    }


def battin_stm(
    r1_vec: np.ndarray, v1_vec: np.ndarray, dt: float, prop: dict, mu: float = MU_EARTH
) -> tuple[np.ndarray, dict]:
    """Battin's two-body STM from the universal functions U1-U5.

    Arora, Russell, Strange, and Ottesen (2015), Section IV.A, Eqs. 38-50.
    """
    r2_vec, v2_vec = prop["r2"], prop["v2"]
    f, g, f_dot, g_dot = (prop[key] for key in ("f", "g", "f_dot", "g_dot"))
    r1, r2 = np.linalg.norm(r1_vec), np.linalg.norm(r2_vec)
    root_mu = np.sqrt(mu)
    alpha = 2.0 / r1 - v1_vec @ v1_vec / mu
    sigma1 = r1_vec @ v1_vec / root_mu
    sigma2 = r2_vec @ v2_vec / root_mu
    chi = alpha * root_mu * dt + sigma2 - sigma1
    u1 = -r2 * r1 * f_dot / root_mu
    u2 = r1 * (1.0 - f)
    u3 = root_mu * (dt - g)
    u4 = u1 * u3 - 0.5 * (u2**2 - alpha * u3**2)
    u5 = (chi**3 / 6.0 - u3) / alpha
    c_bar = (3.0 * u5 - chi * u4 - root_mu * dt * u2) / root_mu

    eye, outer = np.eye(3), np.outer
    dv = v2_vec - v1_vec
    block_a = (
        r2 / mu * outer(dv, dv)
        + (r1 * (1.0 - f) * outer(r2_vec, r1_vec) + c_bar * outer(v2_vec, r1_vec))
        / r1**3
```

```
275             + f * eye
276         )
277         block_b = (
278             r1 / mu * (1.0 - f) * (outer(r2_vec - r1_vec, v1_vec) - outer(dv, r1_vec))
279             + c_bar / mu * outer(v2_vec, v1_vec)
280             + g * eye
281         )
282         rotation = outer(r2_vec, v2_vec) - outer(v2_vec, r2_vec)
283         block_c = (
284             -outer(dv, r1_vec) / r1**2
285             - outer(r2_vec, dv) / r2**2
286             + f_dot
287             * (
288                 eye
289                 - outer(r2_vec, r2_vec) / r2**2
290                 + outer(rotation @ r2_vec, dv) / (mu * r2)
291             )
292             - mu * c_bar / (r1**3 * r2**3) * outer(r2_vec, r1_vec)
293         )
294         block_d = (
295             r1 / mu * outer(dv, dv)
296             + (r1 * (1.0 - f) * outer(r2_vec, r1_vec) - c_bar * outer(r2_vec, v1_vec))
297             / r2**3
298             + g_dot * eye
299         )
300         terms = {
301             "alpha": alpha,
302             "sigma1": sigma1,
303             "sigma2": sigma2,
304             "chi": chi,
305             "U1": u1,
306             "U2": u2,
307             "U3": u3,
308             "U4": u4,
309             "U5": u5,
310             "C_bar": c_bar,
311         }
312         return np.block([[block_a, block_b], [block_c, block_d]]), terms
313
314
315     def finite_difference_stm(
316         r1_vec: np.ndarray, v1_vec: np.ndarray, dt: float, mu: float = MU_EARTH
317     ) -> np.ndarray:
318         """Central-difference Jacobian of the Lagrange propagation (checks Phi)."""
319         state = np.concatenate((r1_vec, v1_vec))
320         steps = np.array([1e-3] * 3 + [1e-6] * 3)  # 1 m and 1 mm/s
```

```
321       phi = np.empty((6, 6))
322       for k, step in enumerate(steps):
323           offset = np.zeros(6)
324           offset[k] = step
325           ends = []
326           for sign in (1.0, -1.0):
327               shifted = state + sign * offset
328               prop = lagrange_propagate(shifted[:3], shifted[3:], dt, mu)
329               ends.append(np.concatenate((prop["r2"], prop["v2"])))
330           phi[:, k] = (ends[0] - ends[1]) / (2.0 * step)
331       return phi
332
333
334   def block_errors(difference: np.ndarray, reference: np.ndarray) -> list[float]:
335       """Largest entry of each 3x3 block of a difference, relative to the block."""
336       errors = []
337       for rows in (slice(0, 3), slice(3, 6)):
338           for cols in (slice(0, 3), slice(3, 6)):
339               errors.append(
340                   np.max(np.abs(difference[rows, cols]))
341                   / np.max(np.abs(reference[rows, cols]))
342               )
343       return errors
344
345
346   def lambert_prussing(
347       r1_vec: np.ndarray,
348       r2_vec: np.ndarray,
349       dt: float,
350       shortway: bool = True,
351       mu: float = MU_EARTH,
352   ) -> dict:
353       """Zero-revolution Lambert solution by iterating on the semimajor axis.
354
355       Lagrange's time equation in the alpha, beta form (Prussing and Conway):
356       s = (r1 + r2 + c)/2, sin^2(alpha/2) = s/2a, sin^2(beta/2) = (s - c)/2a.
357       """
358       r1, r2 = np.linalg.norm(r1_vec), np.linalg.norm(r2_vec)
359       chord = np.linalg.norm(r2_vec - r1_vec)
360       s = (r1 + r2 + chord) / 2.0
361       sign = 1.0 if shortway else -1.0
362       # Parabolic (Euler) flight time; shorter flights are hyperbolic.
363       t_p = ((r1 + r2 + chord) ** 1.5 - sign * (r1 + r2 - chord) ** 1.5) / (
364           6.0 * np.sqrt(mu)
365       )
366       if dt <= t_p:
```

```
367             raise ValueError("Flight time is below the parabolic time (hyperbola).")
368         # Minimum-energy ellipse: a_min = s/2, alpha = pi.
369         a_min = s / 2.0
370         beta_c = sign * 2.0 * np.arcsin(np.sqrt((s - chord) / s))
371         t_c = (np.pi - (beta_c - np.sin(beta_c))) / np.sqrt(mu / a_min**3)
372 
373         def angles(a: float) -> tuple[float, float]:
374             alpha = 2.0 * np.arcsin(min(1.0, np.sqrt(s / (2.0 * a))))
375             beta = sign * 2.0 * np.arcsin(np.sqrt((s - chord) / (2.0 * a)))
376             # Flights longer than t_c take the branch through the vacant focus.
377             return (TAU - alpha if dt > t_c else alpha), beta
378 
379         def residual(a: float) -> float:
380             alpha, beta = angles(a)
381             return (alpha - beta - (np.sin(alpha) - np.sin(beta))) / np.sqrt(mu / a**3) - dt
382 
383         # Bracket the root in a, then bisect.
384         low, high = a_min, 2.0 * a_min
385         while np.sign(residual(high)) == np.sign(residual(low)):
386             high *= 2.0
387         for iterations in range(1, 201):
388             middle = 0.5 * (low + high)
389             if np.sign(residual(middle)) == np.sign(residual(low)):
390                 low = middle
391             else:
392                 high = middle
393             if high - low <= 1e-14 * high:
394                 break
395         a = 0.5 * (low + high)
396 
397         alpha, beta = angles(a)
398         delta = alpha - beta
399         f = 1.0 - a / r1 * (1.0 - np.cos(delta))
400         g = dt - np.sqrt(a**3 / mu) * (delta - np.sin(delta))
401         g_dot = 1.0 - a / r2 * (1.0 - np.cos(delta))
402         swept = np.arccos(r1_vec @ r2_vec / (r1 * r2))
403         return {
404             "c": chord,
405             "s": s,
406             "a_min": a_min,
407             "t_p": t_p,
408             "beta_c": beta_c,
409             "t_c": t_c,
410             "a": a,
411             "alpha": alpha,
412             "beta": beta,
```

```
413             "dE": delta,
414             "dtheta": swept if shortway else TAU - swept,
415             "f": f,
416             "g": g,
417             "g_dot": g_dot,
418             "v1": (r2_vec - f * r1_vec) / g,
419             "v2": (g_dot * r2_vec - r1_vec) / g,
420             "iterations": iterations,
421             "residual": residual(a),
422         }
423 
424 
425 def problem_01() -> None:
426     """Curtis 2.45: delta-theta Lagrange coefficients over a 10 min coast."""
427     heading("p01")
428     r0_vec = np.array([6320.0, 0.0, 7750.0])  # km
429     v0_vec = np.array([0.0, 11.0, 0.0])  # km/s
430     dt = 10.0 * 60.0  # s
431 
432     r0 = np.linalg.norm(r0_vec)
433     v0 = np.linalg.norm(v0_vec)
434     vr0 = r0_vec @ v0_vec / r0
435     h_vec = np.cross(r0_vec, v0_vec)
436     h = np.linalg.norm(h_vec)
437     h_hat = h_vec / h
438     e_vec = np.cross(v0_vec, h_vec) / MU_EARTH - r0_vec / r0
439     e = np.linalg.norm(e_vec)
440     energy = v0**2 / 2.0 - MU_EARTH / r0
441     if e <= 1.0 or energy <= 0.0:
442         raise ValueError("Expected a hyperbolic trajectory.")
443 
444     # Signed initial true anomaly; v_r0 = 0 places the spacecraft at perigee.
445     theta0 = np.arctan2(h_hat @ np.cross(e_vec, r0_vec), e_vec @ r0_vec)
446 
447     # Hyperbolic Kepler's equation turns the 10 min coast into delta theta.
448     ratio = np.sqrt((e + 1.0) / (e - 1.0))
449     anomaly0 = 2.0 * np.arctanh(np.tan(theta0 / 2.0) / ratio)
450     mean_rate = MU_EARTH**2 / h**3 * (e * e - 1.0) ** 1.5  # rad/s
451     mean_anomaly = e * np.sinh(anomaly0) - anomaly0 + mean_rate * dt
452     anomaly = solve_hyperbolic_kepler(e, mean_anomaly)
453     theta = 2.0 * np.arctan(ratio * np.tanh(anomaly / 2.0))
454     dtheta = theta - theta0
455 
456     # Lagrange coefficients in terms of delta theta (Curtis Eqs. 2.152, 2.158).
457     c, s = np.cos(dtheta), np.sin(dtheta)
458     r = (h**2 / MU_EARTH) / (
```

```
459             1.0 + (h**2 / (MU_EARTH * r0) - 1.0) * c - h * vr0 / MU_EARTH * s
460         )
461         f = 1.0 - MU_EARTH * r / h**2 * (1.0 - c)
462         g = r * r0 / h * s
463         f_dot = (
464             MU_EARTH
465             / h
466             * (1.0 - c)
467             / s
468             * (MU_EARTH / h**2 * (1.0 - c) - 1.0 / r0 - 1.0 / r)
469         )
470         g_dot = 1.0 - MU_EARTH * r0 / h**2 * (1.0 - c)
471         r_vec = f * r0_vec + g * v0_vec
472         v_vec = f_dot * r0_vec + g_dot * v0_vec
473
474         # Independent checks: invariants, the Lagrange identity, the swept angle, and
475         # direct numerical integration of the two-body equations.
476         np.testing.assert_allclose(np.linalg.norm(r_vec), r, rtol=1e-12)
477         np.testing.assert_allclose(f * g_dot - f_dot * g, 1.0, atol=1e-12)
478         np.testing.assert_allclose(np.cross(r_vec, v_vec), h_vec, rtol=0.0, atol=1e-7)
479         np.testing.assert_allclose(v_vec @ v_vec / 2.0 - MU_EARTH / r, energy, rtol=1e-12)
480         swept = np.arctan2(h_hat @ np.cross(r0_vec, r_vec), r0_vec @ r_vec)
481         np.testing.assert_allclose(swept, dtheta, atol=1e-12)
482         numerical = rk4(two_body_rate, np.concatenate((r0_vec, v0_vec)), dt, 6000)
483         rk4_error = np.linalg.norm(numerical[:3] - r_vec)
484         # The angle to the integrated position checks delta theta without f and g.
485         r_numerical = numerical[:3]
486         cos_numerical = r0_vec @ r_numerical / (r0 * np.linalg.norm(r_numerical))
487         swept_numerical = np.arctan2(
488             h_hat @ np.cross(r0_vec, r_numerical), r0_vec @ r_numerical
489         )
490         np.testing.assert_allclose(swept_numerical, dtheta, atol=1e-9)
491         assert rk4_error < 1e-6, rk4_error
492
493         write_output(
494             "01a",
495             [
496                 f"r0 = {r0:.6f} km",
497                 f"v0 = {v0:.6f} km/s",
498                 f"vr0 = {clean(vr0):.6f} km/s",
499                 *vector_lines("h", h_vec, "km^2/s"),
500                 f"h = {h:.6f} km^2/s",
501                 *vector_lines("e", e_vec, precision=10),
502                 f"e = {e:.10f}",
503                 f"energy = {energy:.6f} km^2/s^2",
504                 f"theta0 = {np.degrees(clean(theta0)):.6f} deg",
```

```
505                 f"Mh rate = {mean_rate:.10f} rad/s",
506                 f"M_h = {mean_anomaly:.10f}",
507                 f"F = {anomaly:.10f}",
508                 f"theta = {np.degrees(theta):.6f} deg",
509                 f"dtheta = {np.degrees(dtheta):.6f} deg",
510                 f"r = {r:.6f} km",
511                 f"f = {f:.10f}",
512                 f"g = {g:.6f} s",
513                 f"f_dot = {f_dot:.10f} 1/s",
514                 f"g_dot = {g_dot:.10f}",
515                 *vector_lines("r", r_vec, "km"),
516                 *vector_lines("v", v_vec, "km/s"),
517                 f"lagrange identity = {f * g_dot - f_dot * g:.12f}",
518                 f"rk4 position difference = {rk4_error:.3e} km",
519             ],
520         )
521         write_output(
522             "01b",
523             [
524                 f"theta0 = {np.degrees(clean(theta0)):.6f} deg",
525                 f"theta = {np.degrees(theta):.6f} deg",
526                 f"dtheta = {np.degrees(dtheta):.6f} deg",
527                 *vector_lines("r_rk4", r_numerical, "km"),
528                 f"cos dtheta rk4 = {cos_numerical:.10f}",
529                 f"swept rk4 = {np.degrees(swept_numerical):.6f} deg",
530             ],
531         )
532
533         # Trajectory in the perifocal frame (p toward perigee, q along the motion).
534         p_hat = e_vec / e
535         q_hat = np.cross(h_hat, p_hat)
536         semilatus = h**2 / MU_EARTH
537
538         def perifocal(angles: np.ndarray) -> np.ndarray:
539             radius = semilatus / (1.0 + e * np.cos(angles))
540             return np.array([radius * np.cos(angles), radius * np.sin(angles)]) / 1e3
541
542         fig, ax = plt.subplots(figsize=(5.4, 5.0))
543         ax.add_patch(
544             plt.Circle((0.0, 0.0), R_EARTH / 1e3, facecolor=GRID, edgecolor=BASELINE)
545         )
546         ax.text(0.0, -3.2, "Earth", ha="center", va="center", color=INK_SECONDARY)
547         branch = perifocal(np.radians(np.linspace(-70.0, 70.0, 400)))
548         ax.plot(*branch, color=MUTED, lw=1.0, ls="--", label="hyperbolic trajectory")
549         arc = perifocal(np.linspace(theta0, theta, 200))
550         ax.plot(*arc, color=SERIES[0], lw=2.5, label="10 min coast")
```

```
    for vector, text, offset in (
        (r0_vec, r"$\mathbf{r}_0$ ($t = 0$, perigee)", (6, -12)),
        (r_vec, r"$\mathbf{r}$ ($t = 10$ min)", (6, 4)),
    ):
        tip = np.array([vector @ p_hat, vector @ q_hat]) / 1e3
        ax.annotate(
            "",
            xy=tip,
            xytext=(0.0, 0.0),
            arrowprops={"arrowstyle": "-|>", "color": INK_SECONDARY, "lw": 1.2},
        )
        ax.annotate(text, tip, xytext=offset, textcoords="offset points", color=INK)
    wedge = np.linspace(theta0, theta, 100)
    ax.plot(8.0 * np.cos(wedge), 8.0 * np.sin(wedge), color=SERIES[1], lw=1.5)
    middle = (theta0 + theta) / 2.0
    ax.text(
        10.9 * np.cos(middle),
        10.9 * np.sin(middle),
        rf"$\Delta\theta = {np.degrees(dtheta):.2f}^\circ$",
        color=INK,
        ha="left",
        va="center",
    )
    ax.set_aspect("equal")
    ax.set_xlim(-8.0, 16.0)
    ax.set_ylim(-14.0, 14.0)
    ax.set_xlabel(r"$\hat{\mathbf{p}}$ (toward perigee) [$10^3$ km]")
    ax.set_ylabel(r"$\hat{\mathbf{q}}$ [$10^3$ km]")
    ax.legend(loc="lower left")
    save_figure(fig, "01a")


def problem_02() -> None:
    """Curtis 7.7: CW propagation of a satellite near a space station."""
    heading("p02")
    period = 90.0 * 60.0  # s
    t = 15.0 * 60.0  # s
    n = TAU / period
    dr0 = np.array([1.0, 0.0, 0.0])  # km
    dv0 = np.array([0.0, 10.0, 0.0]) / 1e3  # km/s

    phi_rr, phi_rv, _, _ = cw_stm(n, t)
    dr, dv = cw_propagate(n, t, dr0, dv0)
    distance = np.linalg.norm(dr)
    error = check_cw(n, t, dr0, dv0)

```

```
    write_output(
        "02",
        [
            f"n = {n:.10f} rad/s",
            f"nt = {n * t:.10f} rad",
            f"Phi_rr_11 = {phi_rr[0, 0]:.6f}",
            f"Phi_rr_21 = {phi_rr[1, 0]:.6f}",
            f"Phi_rv_12 = {phi_rv[0, 1]:.6f} s",
            f"Phi_rv_22 = {phi_rv[1, 1]:.6f} s",
            *vector_lines("dr", dr, "km"),
            f"distance = {distance:.6f} km",
            *vector_lines("dv", dv * 1e3, "m/s"),
            f"rk4 state difference = {error:.3e}",
        ],
    )

    times = np.linspace(0.0, t, 301)
    path = np.array([cw_propagate(n, tau, dr0, dv0)[0] for tau in times])
    fig, ax = plt.subplots(figsize=(4.6, 5.4))
    ax.plot(path[:, 1], path[:, 0], color=SERIES[0], lw=2.0)
    ax.plot([0.0, dr[1]], [0.0, dr[0]], color=INK_SECONDARY, lw=1.0, ls="--")
    ax.text(
        dr[1] / 2.0 + 0.3,
        dr[0] / 2.0,
        f"{distance:.2f} km",
        color=INK,
        ha="left",
        va="center",
    )
    ax.plot(0.0, 0.0, marker="s", color=INK, ms=7, ls="none")
    ax.annotate(
        "space station",
        (0.0, 0.0),
        xytext=(-8, -4),
        textcoords="offset points",
        ha="right",
        va="top",
        color=INK,
    )
    ax.plot(dr0[1], dr0[0], "o", mfc="white", mec=SERIES[0], mew=2.0, ms=8)
    ax.annotate(
        r"$t = 0$",
        (dr0[1], dr0[0]),
        xytext=(-8, 0),
        textcoords="offset points",
        ha="right",
```

```
643             va="center",
644             color=INK,
645         )
646         ax.plot(dr[1], dr[0], "o", color=SERIES[0], ms=8)
647         ax.annotate(
648             r"$t = 15$ min",
649             (dr[1], dr[0]),
650             xytext=(8, 0),
651             textcoords="offset points",
652             ha="left",
653             va="center",
654             color=INK,
655         )
656         ax.set_aspect("equal")
657         ax.set_xlim(-4.0, 6.0)
658         ax.set_ylim(-1.0, 12.5)
659         ax.set_xlabel(r"$\delta y$, along-track [km]")
660         ax.set_ylabel(r"$\delta x$, radial [km]")
661         save_figure(fig, "02")
662 
663 
664 def problem_03() -> None:
665     """Curtis 7.8: retrofire by a leading spacecraft in the same circular orbit."""
666     heading("p03")
667     period = 2.0 * 3600.0  # s
668     t = 30.0 * 60.0  # s
669     n = TAU / period
670     dr0 = np.array([0.0, 6.0, 0.0])  # km
671     # Co-orbiting, so B is at rest in A's CW frame until the in-track retrofire.
672     dv0 = np.array([0.0, 0.0, 0.0]) + np.array([0.0, -3.0, 0.0]) / 1e3  # km/s
673 
674     phi_rr, phi_rv, phi_vr, phi_vv = cw_stm(n, t)
675     dr, dv = cw_propagate(n, t, dr0, dv0)
676     distance = np.linalg.norm(dr)
677     speed = np.linalg.norm(dv)
678     error = check_cw(n, t, dr0, dv0)
679     # Before the burn, B is an equilibrium of the CW equations.
680     np.testing.assert_allclose(
681         cw_propagate(n, t, dr0, np.zeros(3))[0], dr0, rtol=0.0, atol=1e-12
682     )
683 
684     write_output(
685         "03",
686         [
687             f"n = {n:.10f} rad/s",
688             f"nt = {n * t:.10f} rad",
```

```
689                 f"Phi_rr_22 = {phi_rr[1, 1]:.6f}",
690                 f"Phi_rv_12 = {phi_rv[0, 1]:.6f} s",
691                 f"Phi_rv_22 = {phi_rv[1, 1]:.6f} s",
692                 f"Phi_vr_12 = {clean(phi_vr[0, 1]):.6f} 1/s",
693                 f"Phi_vr_22 = {clean(phi_vr[1, 1]):.6f} 1/s",
694                 f"Phi_vv_12 = {phi_vv[0, 1]:.6f}",
695                 f"Phi_vv_22 = {clean(phi_vv[1, 1]):.6f}",
696                 *vector_lines("dr", dr, "km"),
697                 f"distance = {distance:.6f} km",
698                 *vector_lines("dv", dv * 1e3, "m/s"),
699                 f"speed = {speed * 1e3:.6f} m/s",
700                 f"rk4 state difference = {error:.3e}",
701             ],
702         )
703
704         times = np.linspace(0.0, t, 301)
705         path = np.array([cw_propagate(n, tau, dr0, dv0)[0] for tau in times])
706         fig, ax = plt.subplots(figsize=(5.4, 4.6))
707         ax.plot(path[:, 1], path[:, 0], color=SERIES[0], lw=2.0)
708         ax.plot([0.0, dr[1]], [0.0, dr[0]], color=INK_SECONDARY, lw=1.0, ls="--")
709         ax.annotate(
710             f"{distance:.2f} km",
711             (dr[1] / 2.0, dr[0] / 2.0),
712             xytext=(-8, -8),
713             textcoords="offset points",
714             color=INK,
715             ha="right",
716             va="top",
717         )
718         ax.plot(0.0, 0.0, marker="s", color=INK, ms=7, ls="none")
719         ax.annotate(
720             r"$A$",
721             (0.0, 0.0),
722             xytext=(-8, 0),
723             textcoords="offset points",
724             ha="right",
725             va="center",
726             color=INK,
727         )
728         ax.plot(dr0[1], dr0[0], "o", mfc="white", mec=SERIES[0], mew=2.0, ms=8)
729         ax.annotate(
730             r"$B$, $t = 0$ (retrofire)",
731             (dr0[1], dr0[0]),
732             xytext=(0, 9),
733             textcoords="offset points",
734             ha="center",
```

```
735             va="bottom",
736             color=INK,
737         )
738         ax.plot(dr[1], dr[0], "o", color=SERIES[0], ms=8)
739         ax.annotate(
740             r"$B$, $t = 30$ min",
741             (dr[1], dr[0]),
742             xytext=(-10, -6),
743             textcoords="offset points",
744             ha="right",
745             va="top",
746             color=INK,
747         )
748         # Relative velocity at 30 min, drawn at 0.25 km per m/s.
749         arrow = 0.25 * dv * 1e3
750         ax.annotate(
751             "",
752             xy=(dr[1] + arrow[1], dr[0] + arrow[0]),
753             xytext=(dr[1], dr[0]),
754             arrowprops={"arrowstyle": "-|>", "color": SERIES[1], "lw": 1.8},
755         )
756         ax.annotate(
757             rf"$\delta\mathbf{{v}}$, {speed * 1e3:.2f} m/s",
758             (dr[1] + arrow[1], dr[0] + arrow[0]),
759             xytext=(6, 0),
760             textcoords="offset points",
761             ha="left",
762             va="center",
763             color=INK,
764         )
765         ax.set_aspect("equal")
766         ax.set_xlim(-1.5, 13.0)
767         ax.set_ylim(-9.0, 1.5)
768         ax.set_xlabel(r"$\delta y$, along-track [km]")
769         ax.set_ylabel(r"$\delta x$, radial [km]")
770         save_figure(fig, "03")
771
772
773     def problem_04() -> None:
774         """Curtis 7.10: relative speed half an orbit after an along-track launch."""
775         heading("p04")
776         # Nondimensional: n = 1 and v = 1, so positions are in v/n and time is nt.
777         n, v = 1.0, 1.0
778         dr0 = np.zeros(3)
779         dv0 = np.array([0.0, v, 0.0])
780
```

```
    dr, dv = cw_propagate(n, np.pi / n, dr0, dv0)
    ratio = np.linalg.norm(dv) / v
    np.testing.assert_allclose(dv, [0.0, -7.0 * v, 0.0], atol=1e-12)
    check_cw(n, np.pi / n, dr0, dv0)

    angles = np.linspace(0.0, np.pi, 301)
    states = [cw_propagate(n, angle / n, dr0, dv0) for angle in angles]
    path = np.array([state[0] for state in states]) * n / v
    speeds = np.array([np.linalg.norm(state[1]) for state in states]) / v
    closed_form = np.sqrt(13.0 - 24.0 * np.cos(angles) + 12.0 * np.cos(angles) ** 2)
    np.testing.assert_allclose(speeds, closed_form, rtol=1e-12)
    assert np.argmax(speeds) == len(angles) - 1

    write_output(
        "04",
        [
            f"nt = {np.pi:.10f} rad",
            *vector_lines("dr", dr * n / v),
            *vector_lines("dv", dv / v),
            f"speed ratio = {ratio:.6f}",
        ],
    )

    fig, (left, right) = plt.subplots(1, 2, figsize=(8.0, 3.4), width_ratios=(1.4, 1.0))
    left.plot(path[:, 1], path[:, 0], color=SERIES[0], lw=2.0)
    left.plot(0.0, 0.0, "o", mfc="white", mec=SERIES[0], mew=2.0, ms=8)
    left.annotate(
        r"$nt = 0$, $\delta\mathbf{v}_0 = v\,\hat{\mathbf{j}}$",
        (0.0, 0.0),
        xytext=(-6, -8),
        textcoords="offset points",
        ha="right",
        va="top",
        color=INK,
    )
    end = dr * n / v
    left.plot(end[1], end[0], "o", color=SERIES[0], ms=8)
    left.annotate(
        r"$nt = \pi$",
        (end[1], end[0]),
        xytext=(0, 9),
        textcoords="offset points",
        ha="center",
        va="bottom",
        color=INK,
    )
```

```
827       # Relative velocity at nt = pi, drawn at 0.5 (v/n) per v.
828       tip = (end[1] + 0.5 * dv[1] / v, end[0] + 0.5 * dv[0] / v)
829       left.annotate(
830           "",
831           xy=tip,
832           xytext=(end[1], end[0]),
833           arrowprops={"arrowstyle": "-|>", "color": SERIES[1], "lw": 1.8},
834       )
835       left.annotate(
836           r"$\delta\mathbf{v} = -7v\,\hat{\mathbf{j}}$",
837           tip,
838           xytext=(0, -8),
839           textcoords="offset points",
840           ha="left",
841           va="top",
842           color=INK,
843       )
844       left.set_aspect("equal")
845       left.set_xlim(-13.5, 1.5)
846       left.set_ylim(-2.0, 5.5)
847       left.set_xlabel(r"$n\,\delta y / v$, along-track")
848       left.set_ylabel(r"$n\,\delta x / v$, radial")
849
850       right.plot(angles, speeds, color=SERIES[0], lw=2.0)
851       right.plot(np.pi, ratio, "o", color=SERIES[0], ms=8)
852       right.annotate(
853           rf"$|\delta\mathbf{{v}}| = {ratio:.0f}v$",
854           (np.pi, ratio),
855           xytext=(-6, 6),
856           textcoords="offset points",
857           ha="right",
858           va="bottom",
859           color=INK,
860       )
861       right.set_xticks([0.0, np.pi / 2.0, np.pi], ["0", r"$\pi/2$", r"$\pi$"])
862       right.set_ylim(0.0, 8.0)
863       right.set_xlabel(r"$nt$ [rad]")
864       right.set_ylabel(r"$|\delta\mathbf{v}| / v$")
865       fig.tight_layout()
866       save_figure(fig, "04")
867
868
869   def problem_05() -> None:
870       """Propagation with Lagrange coefficients, Battin's STM, and Tellurion."""
871       heading("p05")
872       r1_vec = np.array([5740.13268349, 3314.06715, 0.0])  # km
```

```
    v1_vec = np.array([-2.75082684, 4.76457184, 5.5016536])  # km/s
    dt = 30.0 * 60.0  # s

    # (a) Lagrange coefficients in terms of Delta E.
    prop = lagrange_propagate(r1_vec, v1_vec, dt)
    r2_vec, v2_vec = prop["r2"], prop["v2"]
    energy = v1_vec @ v1_vec / 2.0 - MU_EARTH / np.linalg.norm(r1_vec)
    np.testing.assert_allclose(np.linalg.norm(r2_vec), prop["r"], rtol=1e-12)
    np.testing.assert_allclose(
        prop["f"] * prop["g_dot"] - prop["f_dot"] * prop["g"], 1.0, atol=1e-12
    )
    np.testing.assert_allclose(
        np.cross(r2_vec, v2_vec), np.cross(r1_vec, v1_vec), rtol=1e-12
    )
    np.testing.assert_allclose(
        v2_vec @ v2_vec / 2.0 - MU_EARTH / prop["r"], energy, rtol=1e-12
    )
    numerical = rk4(two_body_rate, np.concatenate((r1_vec, v1_vec)), dt, 6000)
    rk4_error = np.linalg.norm(numerical[:3] - r2_vec)
    assert rk4_error < 1e-6, rk4_error
    period = TAU * np.sqrt(prop["a"] ** 3 / MU_EARTH)

    write_output(
        "05a",
        [
            f"r1 = {np.linalg.norm(r1_vec):.6f} km",
            f"v1 = {np.linalg.norm(v1_vec):.6f} km/s",
            f"energy = {energy:.6f} km^2/s^2",
            f"a = {prop['a']:.6f} km",
            f"e = {prop['e']:.10f}",
            f"period = {period:.6f} s",
            f"n = {np.sqrt(MU_EARTH / prop['a'] ** 3):.10f} rad/s",
            f"sigma1 = {prop['sigma1']:.10f} km^0.5",
            f"ecos = {prop['ecos']:.10f}",
            f"esin = {prop['esin']:.10f}",
            f"M = {prop['M']:.10f} rad",
            f"dE = {prop['dE']:.10f} rad",
            f"dE_deg = {np.degrees(prop['dE']):.6f} deg",
            f"r2 = {prop['r']:.6f} km",
            f"f = {prop['f']:.10f}",
            f"g = {prop['g']:.6f} s",
            f"f_dot = {prop['f_dot']:.10f} 1/s",
            f"g_dot = {prop['g_dot']:.10f}",
            *vector_lines("r2", r2_vec, "km"),
            *vector_lines("v2", v2_vec, "km/s", precision=9),
            f"dt = {dt:.1f} s",
```

```
            f"lagrange identity = {prop['f'] * prop['g_dot'] - prop['f_dot'] *
            prop['g']:.12f}",
            f"rk4 position difference = {sci(rk4_error, 3)} km",
        ],
    )

    # (b) Battin's two-body STM in universal functions.
    phi, terms = battin_stm(r1_vec, v1_vec, dt, prop)
    fd_errors = block_errors(finite_difference_stm(r1_vec, v1_vec, dt) - phi, phi)
    assert max(fd_errors) < 1e-6, fd_errors
    # A two-body STM is symplectic: Phi^T J Phi = J, so det(Phi) = 1.
    zero, eye = np.zeros((3, 3)), np.eye(3)
    j_matrix = np.block([[zero, eye], [-eye, zero]])
    symplectic_error = np.max(np.abs(phi.T @ j_matrix @ phi - j_matrix))
    assert symplectic_error < 1e-9, symplectic_error
    determinant = np.linalg.det(phi)
    np.testing.assert_allclose(determinant, 1.0, atol=1e-9)
    np.testing.assert_allclose(
        terms["chi"], np.sqrt(prop["a"]) * prop["dE"], rtol=1e-12
    )

    write_output(
        "05b",
        [
            f"alpha = {terms['alpha']:.12f} 1/km",
            f"sigma1 = {terms['sigma1']:.10f} km^0.5",
            f"sigma2 = {terms['sigma2']:.10f} km^0.5",
            f"chi = {terms['chi']:.10f} km^0.5",
            f"U1 = {terms['U1']:.6f} km^0.5",
            f"U2 = {terms['U2']:.6f} km",
            f"U3 = {terms['U3']:.6f} km^1.5",
            f"U4 = {terms['U4']:.6f} km^2",
            f"U5 = {terms['U5']:.6f} km^2.5",
            f"C_bar = {terms['C_bar']:.6f} km s",
            *matrix_lines("A", phi[:3, :3], precision=7),
            *matrix_lines("B", phi[:3, 3:], "s", precision=4),
            *matrix_lines("C", phi[3:, :3] * 1e3, "1e-3/s", precision=7),
            *matrix_lines("D", phi[3:, 3:], precision=7),
            f"finite difference error = {sci(max(fd_errors))}",
            f"symplectic error = {sci(symplectic_error)}",
            f"det = {determinant:.12f}",
        ],
    )

    # (c) Tellurion's two-body propagation and STM, then the source of the
    # differences: Orekit's gravitational parameter.
    tell = run_snippet("s05c")
```

```
965       mu_tell = tell["mu"]
966       dr = np.linalg.norm(tell["r2"] - r2_vec)
967       dv = np.linalg.norm(tell["v2"] - v2_vec)
968       dphi = block_errors(tell["stm"] - phi, phi)
969       # Sensitivity of the final state to mu predicts the position difference.
970       step = 1e-3  # km^3/s^2
971       plus = lagrange_propagate(r1_vec, v1_vec, dt, MU_EARTH + step)["r2"]
972       minus = lagrange_propagate(r1_vec, v1_vec, dt, MU_EARTH - step)["r2"]
973       predicted = (plus - minus) / (2.0 * step) * (mu_tell - MU_EARTH)
974       prop_tell = lagrange_propagate(r1_vec, v1_vec, dt, mu_tell)
975       phi_tell, _ = battin_stm(r1_vec, v1_vec, dt, prop_tell, mu_tell)
976       dr_same = np.linalg.norm(tell["r2"] - prop_tell["r2"])
977       dv_same = np.linalg.norm(tell["v2"] - prop_tell["v2"])
978       dphi_same = block_errors(tell["stm"] - phi_tell, phi_tell)
979       np.testing.assert_allclose(tell["r2"] - r2_vec, predicted, atol=1e-6)
980       assert dr_same < 1e-6 and dv_same < 1e-9, (dr_same, dv_same)
981       assert max(dphi_same) < 1e-8, dphi_same
982
983       write_output(
984           "05c-cmp",
985           [
986               f"mu tellurion = {mu_tell:.4f} km^3/s^2",
987               f"mu difference = {mu_tell - MU_EARTH:.4f} km^3/s^2",
988               f"mu ratio = {sci((mu_tell - MU_EARTH) / MU_EARTH, 2)}",
989               *vector_lines("dr", (tell["r2"] - r2_vec) * 1e3, "m", precision=4),
990               f"dr = {dr * 1e3:.4f} m",
991               f"dv = {dv * 1e6:.4f} mm/s",
992               *vector_lines("predicted", predicted * 1e3, "m", precision=4),
993               f"dphi A = {sci(dphi[0])}",
994               f"dphi B = {sci(dphi[1])}",
995               f"dphi C = {sci(dphi[2])}",
996               f"dphi D = {sci(dphi[3])}",
997               f"dphi max = {sci(max(dphi))}",
998               f"dr same mu = {sci(dr_same * 1e3)} m",
999               f"dv same mu = {sci(dv_same * 1e6)} mm/s",
1000              f"dphi same mu = {sci(max(dphi_same))}",
1001          ],
1002      )
1003
1004
1005  def problem_06() -> None:
1006      """Zero-revolution Lambert problem, both directions."""
1007      heading("p06")
1008      r1_vec = np.array([-6991.0, -5118.0, 7078.0])  # km
1009      r2_vec = np.array([11326.0, -6317.0, -6677.0])  # km
1010      dt = 5200.0  # s
```

```
    r1, r2 = np.linalg.norm(r1_vec), np.linalg.norm(r2_vec)
    branches = {"short": True, "long": False}

    # (a) Semimajor-axis iteration, then the Delta E Lagrange coefficients.
    solutions = {
        branch: lambert_prussing(r1_vec, r2_vec, dt, shortway)
        for branch, shortway in branches.items()
    }
    lines = [
        f"r1 = {r1:.6f} km",
        f"r2 = {r2:.6f} km",
        f"c = {solutions['short']['c']:.6f} km",
        f"s = {solutions['short']['s']:.6f} km",
        f"a_min = {solutions['short']['a_min']:.6f} km",
        f"cos dtheta = {r1_vec @ r2_vec / (r1 * r2):.10f}",
        f"h_z sign = {np.sign(np.cross(r1_vec, r2_vec)[2]):+.0f}",
    ]
    for branch, sol in solutions.items():
        # The solved initial state must reach r2 after dt (Lagrange and RK4).
        prop = lagrange_propagate(r1_vec, sol["v1"], dt)
        np.testing.assert_allclose(prop["r2"], r2_vec, rtol=0.0, atol=1e-6)
        np.testing.assert_allclose(prop["v2"], sol["v2"], rtol=0.0, atol=1e-9)
        numerical = rk4(two_body_rate, np.concatenate((r1_vec, sol["v1"])), dt, 20000)
        assert np.linalg.norm(numerical[:3] - r2_vec) < 1e-6
        np.testing.assert_allclose(prop["a"], sol["a"], rtol=1e-12)
        lines += [
            f"t_p_{branch} = {sol['t_p']:.6f} s",
            f"beta_c_{branch} = {np.degrees(sol['beta_c']):.6f} deg",
            f"t_c_{branch} = {sol['t_c']:.6f} s",
            f"dtheta_{branch} = {np.degrees(sol['dtheta']):.6f} deg",
            f"a_{branch} = {sol['a']:.6f} km",
            f"alpha_{branch} = {np.degrees(sol['alpha']):.6f} deg",
            f"beta_{branch} = {np.degrees(sol['beta']):.6f} deg",
            f"dE_{branch} = {np.degrees(sol['dE']):.6f} deg",
            f"f_{branch} = {sol['f']:.10f}",
            f"g_{branch} = {sol['g']:.6f} s",
            f"g_dot_{branch} = {sol['g_dot']:.10f}",
            *vector_lines(f"v1_{branch}", sol["v1"], "km/s", precision=9),
            *vector_lines(f"v2_{branch}", sol["v2"], "km/s", precision=9),
            f"iterations_{branch} = {sol['iterations']}",
            f"residual_{branch} = {sci(abs(sol['residual']))} s",
        ]
    write_output("06a", lines)

    # (b) Closest approach on each arc: perigee if the arc passes through it,
    # otherwise the nearer endpoint.
```

```
    lines = []
    arcs = {}
    times = np.linspace(0.0, dt, 2001)
    for branch, sol in solutions.items():
        h_vec = np.cross(r1_vec, sol["v1"])
        h_hat = h_vec / np.linalg.norm(h_vec)
        e_vec = np.cross(sol["v1"], h_vec) / MU_EARTH - r1_vec / r1
        e = np.linalg.norm(e_vec)
        theta1 = np.arctan2(h_hat @ np.cross(e_vec, r1_vec), e_vec @ r1_vec) % TAU
        theta2 = theta1 + sol["dtheta"]
        perigee = sol["a"] * (1.0 - e)
        through_perigee = theta2 >= TAU
        r_min = perigee if through_perigee else min(r1, r2)
        # Time from r1 to perigee, from the mean anomaly at r1.
        e1 = 2.0 * np.arctan(np.sqrt((1.0 - e) / (1.0 + e)) * np.tan(theta1 / 2.0))
        mean1 = (e1 - e * np.sin(e1)) % TAU
        t_perigee = (TAU - mean1) / np.sqrt(MU_EARTH / sol["a"] ** 3)
        # Sampled check on the closest approach along the arc.
        arc = np.array([lagrange_propagate(r1_vec, sol["v1"], t)["r2"] for t in times])
        arcs[branch] = arc
        sampled = np.min(np.linalg.norm(arc, axis=1))
        assert r_min <= sampled + 1e-6 and sampled - r_min < 1.0, (r_min, sampled)
        assert through_perigee == (t_perigee < dt)
        lines += [
            f"e_{branch} = {e:.10f}",
            f"r_p_{branch} = {perigee:.6f} km",
            f"theta1_{branch} = {np.degrees(theta1):.6f} deg",
            f"theta2_{branch} = {np.degrees(theta2):.6f} deg",
            f"through perigee_{branch} = {through_perigee}",
            f"t_perigee_{branch} = {t_perigee:.6f} s",
            f"r_min_{branch} = {r_min:.6f} km",
            f"altitude_{branch} = {r_min - R_EARTH:.6f} km",
            f"sampled r_min_{branch} = {sampled:.6f} km",
        ]
    write_output("06b", lines)

    # Both arcs in the transfer plane (x along r1, z along r1 x r2).
    x_hat = r1_vec / r1
    z_hat = np.cross(r1_vec, r2_vec)
    z_hat /= np.linalg.norm(z_hat)
    y_hat = np.cross(z_hat, x_hat)
    fig, ax = plt.subplots(figsize=(5.6, 5.2))
    ax.add_patch(
        plt.Circle((0.0, 0.0), R_EARTH / 1e3, facecolor=GRID, edgecolor=BASELINE)
    )
    ax.text(0.0, 0.0, "Earth", ha="center", va="center", color=INK_SECONDARY)
```

```
    for (branch, arc), color in zip(arcs.items(), SERIES):
        x, y = arc @ x_hat / 1e3, arc @ y_hat / 1e3
        ax.plot(x, y, color=color, lw=2.0, label=f"{branch} way")
        middle = len(x) // 2
        ax.annotate(
            "",
            xy=(x[middle + 1], y[middle + 1]),
            xytext=(x[middle - 1], y[middle - 1]),
            arrowprops={"arrowstyle": "-|>", "color": color, "lw": 1.8},
        )
        closest = np.argmin(np.hypot(x, y))
        ax.plot(x[closest], y[closest], "o", color=color, ms=6)
        ax.annotate(
            rf"$r_{{\min}} = {np.hypot(x[closest], y[closest]) * 1e3:.0f}$ km",
            (x[closest], y[closest]),
            xytext=(8, -4) if branch == "long" else (8, -10),
            textcoords="offset points",
            ha="left",
            va="top",
            color=INK,
        )
    for vector, text, offset in (
        (r1_vec, r"$\mathbf{r}_1$ ($t = 0$)", (8, 0)),
        (r2_vec, r"$\mathbf{r}_2$ ($t = 5200$ s)", (8, 0)),
    ):
        point = (vector @ x_hat / 1e3, vector @ y_hat / 1e3)
        ax.plot(*point, "s", color=INK, ms=6)
        ax.annotate(
            text,
            point,
            xytext=offset,
            textcoords="offset points",
            ha="left",
            va="center",
            color=INK,
        )
    ax.set_aspect("equal")
    ax.set_xlabel(r"along $\hat{\mathbf{r}}_1$ [$10^3$ km]")
    ax.set_ylabel(r"in-plane normal [$10^3$ km]")
    ax.legend(loc="lower left")
    save_figure(fig, "06b")

    # (c) Tellurion's Lambert solver (Orekit), compared with the class method.
    tell = run_snippet("s06c")
    mu_tell = tell["mu"]
    lines = [f"mu tellurion = {mu_tell:.4f} km^3/s^2"]
```

```
    dv_all, dv_same_all = [], []
    for branch, shortway in branches.items():
        v1_tell, v2_tell = tell["velocities"][branch]
        same = lambert_prussing(r1_vec, r2_vec, dt, shortway, mu_tell)
        dv1 = np.linalg.norm(v1_tell - solutions[branch]["v1"])
        dv2 = np.linalg.norm(v2_tell - solutions[branch]["v2"])
        dv1_same = np.linalg.norm(v1_tell - same["v1"])
        dv2_same = np.linalg.norm(v2_tell - same["v2"])
        assert max(dv1_same, dv2_same) < 1e-9, (dv1_same, dv2_same)
        dv_all += [dv1, dv2]
        dv_same_all += [dv1_same, dv2_same]
        lines += [
            *vector_lines(
                f"dv1_{branch}", (v1_tell - solutions[branch]["v1"]) * 1e6, "mm/s", 4
            ),
            f"dv1_{branch} = {dv1 * 1e6:.4f} mm/s",
            f"dv2_{branch} = {dv2 * 1e6:.4f} mm/s",
            f"dv1 same mu_{branch} = {sci(dv1_same * 1e6)} mm/s",
            f"dv2 same mu_{branch} = {sci(dv2_same * 1e6)} mm/s",
            f"da mu_{branch} = {same['a'] - solutions[branch]['a']:.6f} km",
        ]
    write_output(
        "06c-cmp",
        lines
        + [
            f"dv max = {max(dv_all) * 1e6:.4f} mm/s",
            f"dv same mu max = {sci(max(dv_same_all) * 1e6)} mm/s",
        ],
    )

    # (d) Tellurion's analytic two-body propagation of each Lambert state.
    tell = run_snippet("s06d")
    lines = []
    for branch in branches:
        miss = tell["misses"][branch]
        assert miss < 1e-3, miss
        # Mixing gravitational parameters: our mu = 398600 solution flown with
        # Orekit's mu misses r2 by the size of the mu sensitivity.
        mixed = lagrange_propagate(r1_vec, solutions[branch]["v1"], dt, mu_tell)
        lines += [
            f"miss_{branch} = {sci(miss)} m",
            f"mixed miss_{branch} = {np.linalg.norm(mixed['r2'] - r2_vec) * 1e3:.4f} m",
        ]
    write_output("06d-cmp", lines)
```

```
1195 def main() -> None:
1196     #########
1197     # p01 #
1198     #########
1199     problem_01()
1200 
1201     #########
1202     # p02 #
1203     #########
1204     problem_02()
1205 
1206     #########
1207     # p03 #
1208     #########
1209     problem_03()
1210 
1211     #########
1212     # p04 #
1213     #########
1214     problem_04()
1215 
1216     #########
1217     # p05 #
1218     #########
1219     problem_05()
1220 
1221     #########
1222     # p06 #
1223     #########
1224     problem_06()
1225 
1226 
1227 if __name__ == "__main__":
1228     main()
```